

# ARCHITETTURA DEGLI ELABORATORI

## CONVERSIONE DEI NUMERI

### Parte Intera

- divido per n e annoto il resto
- se resto  $\neq 0$ , tolgo e divido nuovamente
- se risultato = 0, ho finito
- Leggo dall'ultimo al primo

### Parte Decimale

- moltiplico la parte decimale per n
- Segno la parte intera
- Moltiplico i decimali nuovamente
- Se decimali = 0, ho finito
- Leggo in ordine

### Inverso:

100011,10011  $\rightarrow$  35,59375

$$1 * 2^5 + 0 * 2^4 + 0 * 2^3 + 0 * 2^2 + 1 * 2^1 + 1 * 2^0 = 35$$

$$1 * 2^{-1} + 0 * 2^{-2} + 0 * 2^{-3} + 1 * 2^{-4} + 2^{-5} = 0,59375$$

### Moltiplicazione senza segno per $X=2^m$

Aggiungo a sx tanti 0 quanti m (**SHIFT LOGICO** sinistra)

$$00011 * 2^3 = 00011000$$

### Divisione senza segno per $X=2^m$ (**SHIFT LOGICO** destra)

Tolgo a dx tanti 0 quanti m

$$0001111 / 2^3 = 0001.111$$

## Complemento

Complemento ad 1: sostituisco tutti gli 1 con 0 e viceversa

Complemento a 2:

- Calcolo il complemento ad 1 e sommo 1 a tutte le cifre, ignoro overflow a sinistra  
OPPURE
- Dalla cifra meno significativa, ricopio tutto fino al primo 1 incluso. Inverto tutte le successive

## Numeri con Segno

Rappresentazione in complemento a 2:

- cifra più a sinistra è il segno
  - 0 positivo
  - 1 negativo
- Se positivo: il numero è descritto dalle cifre successive
- Se negativo: eseguo complemento a 2 sul numero e considero negativo

## Massimi e Minimi

$$0 = 000...00$$

$$-1 = 111...11$$

$$\text{Max Positivo} = 011...11$$

$$\text{Min Negativo} = 100...00$$

$$\text{Su N bit: } [-2^{(N-1)} ; 2^{(N-1)} - 1]$$

## OverFlow

Quando durante un calcolo, il numero di bit richiesti per rappresentare il risultato è maggiore di quelli disponibili.

Se una somma non va in overflow, è sicuramente corretta indipendentemente dai segni.

Nella somma di due numeri relativi codificati nella rappresentazione del complemento a 2 su N cifre, si ha overflow quanto:

- Ci sono addendi dello stesso segno e il segno del risultato è diverso dal segno degli addendi
- Il riporto della colonna n-2 alla colonna n-1 ed il riporto dalla colonna n-1 a quella oltre la cifra più significativa sono discordi (uno è 0 e l'altro è 1)

### Moltiplicazione con segno in complemento a 2:

Sposto a sinistra di M posizioni, a patto di non fare overflow. Se cambia il segno, è overflow

### Divisione con segno in complemento a 2:

- Positivo: spostato verso destra di m posizioni

### Estensione da N a M bit con M>N (SHIFT ARITMETICO)

Estendo il numero del segno per tutti gli spazi avanzati

01001000  
0000000000001001000

11101101  
11111111111111101101

## DIMENSIONE DEI DATI

| Type           | RV32 Size Byte | Size Bit | Range                           | Common Uses |
|----------------|----------------|----------|---------------------------------|-------------|
| char           | 1 Byte         | 8 Bit    | -128<br>127                     |             |
| unsigned char  | 1 Byte         | 8 Bit    | 0<br>255                        | ASCII       |
| short          | 2 Byte         | 16 Bit   | -32768<br>32767                 |             |
| unsigned short | 2 Byte         | 16 Bit   |                                 |             |
| int            | 4 Byte         | 32 Bit   | -2,147,483,648<br>2,147,483,647 |             |
| unsigned int   | 4 Byte         | 32 Bit   | 0<br>4,294,967,295              |             |
| long           | 4 Byte         | 32 Bit   | uguale a int in RV32            |             |
| unsigned long  | 4 Byte         | 32 Bit   | uguale a int in RV32            |             |
| long long      | 8 Byte         | 64 Bit   |                                 |             |
| float          | 4 Byte         | 32 Bit   |                                 |             |
| double         | 8 Byte         | 64 Bit   |                                 |             |
| long double    | 16 Byte        | 128 Bit  |                                 |             |

# ARCHITETTURA DI UN ELABORATORE

**Organizzazione a livelli:** di norma una macchina è strutturata su diversi livelli, la maggior parte da 2 o più (noi studiamo in 5).

.0 Macchina - ALU - Transistor

– Direct Execution on HardWare –

- *Rappresenta l'hardware della macchina ed i circuiti*
- *Porte logiche di base costruite da transistor*
- *Combinazioni di porte creano circuito chiamato ALU*
- *Porte, Memorie e Registri, i registri sono connessi alla ALU per formare il percorso dati lungo il quale i dati si spostano. Alle volte gestito da "microprogramma", non il nostro caso*

.1 ISA: Instruction Set Architecture

– Partial Interpretation (operationing system) –

- *Livello del linguaggio macchina*
- *Supporti Architetturali*
- *Queste azioni saranno eseguite direttamente dall'hardware*

.2 Sistema Operativo

– Translation (assembler) –

- *Molte delle operazioni fanno parte del livello ISA*
- *Nuove operazioni più articolate e dipendenti dal sistema operativo (concorrenza ecc)*

.3 Assembly

(Linguaggio Basso Livello)

– Translation (compiler) –

.4 Problem Oriented

(Linguaggio Alto Livello)

## Organizzazione della CPU in una macchina di Von Neumann

Gestione del ciclo Fetch-Decode-Execute

La CPU si compone di diverse parti distinte:

- Unità di controllo
- Unità aritmetico-logica
- Registri: memoria veloce per dati temporanei

Il data Path (percorso dei dati, percorso di computazione dei dati) si compone di

- Registri
- Unità aritmetico-logica
- Alcuni BUS di trasporto

Registri Importanti

- PC: Program Counter
- IR: Instruction Register

Main Memory contiene sia le istruzioni che i dati, tutto in sequenze di bit.

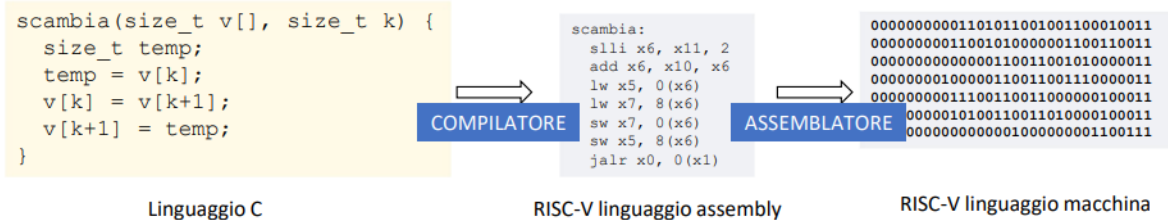
## L'esecuzione delle istruzioni: ciclo di fetch-decode-execute

1. Prendi istruzione seguente dalla memoria e mettila nel IR
2. Cambia PC per indicare l'istruzione seguente
3. Determina il tipo di istruzione letta
4. Se usa dati in memoria, determina dove sono
  - a. Immetti la parola, se necessario, in un registro della CPU
5. Esegui l'istruzione
6. Torna al punto 1 e continua con la prossima

## Il Ruolo del Software

- Organizzazione a livelli
- Software applicativo, ovvero le app usate dal Client finale
- Software di sistema
  - Sistema operativo
  - Compilatore
- Hardware

### Ciclo di vita di un codice per essere eseguito:



### Prestazioni di un codice:

- Tempo di esecuzione / tempo di risposta: tempo tra inizio e completamento programma
- Throughput o larghezza di banda: numero di task eseguiti nell'unità di tempo

### Misurazione Prestazioni:

- Tempo di esecuzione della CPU:
  - tempo di CPU utente: tempo speso a eseguire cose utente
  - tempo di CPU di sistema: tempo speso a eseguire cose del sistema operativo
- Clock: genera un segnale a frequenza costante
- Periodo/Frequenza di di Clock: periodo durata di un ciclo di clock (es. 4 GHZ)
- CPI - Cicli di clock per Istruzione: numero medio di cicli di clock per un istruzione

Tempo di CPU relativo ad un programma:  $N \text{ Cicli di clock} / \text{Frequenza Clock}$

Tempo di CPU:  $\text{Numero Istruzioni} \times \text{CPI} / \text{Frequenza}$

# RISC-V INSTRUCTION SET

RISC: Reduced Instruction Set Computer

Ha come obiettivi:

- costruire un hardware semplice
- Compilatori efficienti
- Massimizzare prestazioni
- Minimizzare i costi
- Minimizzare il consumo energetico

ISA

- ISA semplice ed estendibile
- License-free, royalty-free
- Standard mantenuto dalla RISC-V Foundation
- Specifiche adatte a dispositivi eterogenei (da microcontrollori a supercomputer)
- NON é un azienda, NON è una CPU

Tipologie di Istruzione

- Aritmetiche
  - Notazione rigida: esattamente 3 operandi, ordine fisso degli operandi
  - Addizione:  $a = b + c \rightarrow \text{add } a, b, c$
  - Sottrazione:  $a = b - c \rightarrow \text{sub } a, b, c$
  - Cambio di segno: sottrazione  $0 - n \rightarrow \text{sub } a, x0, a$  (x0 contiene SEMPRE 0)
  - Copiare il valore: somma  $0 + n \rightarrow \text{add } a, x0, b$  (copia b in a)
- Logiche
- Accesso alla memoria
- Condizionali

La traduzione di un codice di alto livello in assembly è svolto dal compilatore.

Un'unica operazione di alto livello potrebbe essere composta da molteplici di assembly.

Parola (word): 32 Bit

Parola doppia (doubleword): 64 Bit

## Registri e Memoria

- **32 registri per interi**
  - Indicati con x0, ..., X31
  - Rappresentati in complemento a due
- 32 registri per numeri in virgola mobile
- registro PC
- 4096 ( $2^{12}$ ) registri control status
- Memoria centrale

| Registro         | Nome           | Utilizzo                                                                               |
|------------------|----------------|----------------------------------------------------------------------------------------|
| x0               | zero           | Costante zero                                                                          |
| x1               | ra             | Return address                                                                         |
| x2               | sp             | Stack pointer                                                                          |
| x3               | gp             | Global pointer                                                                         |
| x4               | tp             | Puntatore a thread                                                                     |
| x8               | s0 / fp        | Frame pointer (il contenuto va preservato se utilizzato dalla procedura chiamata)      |
| x10-x11          | a0-a1          | Passaggio di parametri nelle procedure e valori di ritorno                             |
| x12-x17          | a2-a7          | Passaggio di parametri nelle procedure                                                 |
| x5-x7<br>x28-x31 | t0-t2<br>t3-t6 | Registri temporanei, non salvati in caso di chiamata                                   |
| x9               | s1             | Registri da salvare: il contenuto va preservato se utilizzati dalla procedura chiamata |
| x18-x27          | s2-s11         |                                                                                        |

RISC-V Instruction Set

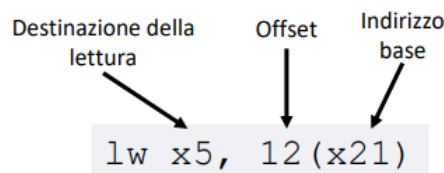
131

## Istruzioni accesso alla Memoria

Se devo processare più dati di quelli che possono avere i registri, carico tutto in ram e poi eseguo mappature e load in registri. Caricare dalla RAM richiede tempo, vogliamo quindi tenere in registro i dati usati di frequente per evitare troppi giri.

La rappresentazione dell'indirizzo in memoria è composto da:

- indirizzo base (registro di mappatura)
- scostamento o offset (*anche negativo, da -2048 a 2047*)



load copia un dato dalla memoria da un registro, è una pseudo istruzione in quanto si articola poi effettivamente in

- lw (load word): copia una parola dalla memoria ad un registro
- ld (load doubleword): copia una parola doppia dalla memoria al registro (**solo x 64bit**)
- li (load immediate): carica un valore immediato
- lb (load Byte): copia un byte al registro
- lh (load Halfword): copia mezza parola al registro
- la: carica l'indirizzo di una variabile in un registro **NON IL CONTENUTO**

store copia un dato da un registro alla memoria, è una pseudo istruzione in quanto si articola poi poi effettivamente in

- sw (store word): copia una parola (4 Byte - 32 Bit) da registro a memoria
- sd (store doubleword): copia una doppia parola (8 Byte - 64 Bit) in memoria
- sb (store Byte): copia un singolo byte (1 Byte - 8 Bit) in memoria
- sh (store Halfword): copia mezza parola (2 Byte - 16 Bit) in memoria

**NOI CONSIDERIAMO SOLO RV32I ovvero max 32 Bit in un registro.**

## Allineamento degli indirizzi

Ogni registro (nel nostro caso) ha dimensione 32 bit (4 byte).

È conveniente che i dati siano allineati ovvero che l'indirizzo del registro sia un multiplo della dimensione dei dati che vogliamo inserire.

```
li t0, 0x10010000    # indirizzo allineato a 4
lw t1, 0(t0)         # OK
```

$0x10010000 = (1 \times 16^7) + (0 \times 16^6) + (0 \times 16^5) + (1 \times 16^4) + (0 \times 16^3) + (0 \times 16^2) + (0 \times 16^1) + (0 \times 16^0)$   
 $= 268435456 + 0 + 0 + 65536 + 0 + 0 + 0 + 0$   
 $= 268500992$   
 $268500992 / 4 = 67125248$                       INTERO, allineato

```
li t0, 0x10010002    # NON allineato a 4
lw t1, 0(t0)         # ERRORE
```

$0x10010002 = (1 \times 16^8) + (0 \times 16^7) + (0 \times 16^6) + (1 \times 16^5) + (0 \times 16^4) + (0 \times 16^3) + (0 \times 16^2) + (0 \times 16^1) + (2 \times 16^0)$   
 $= 4294967296 + 0 + 0 + 1048576 + 0 + 0 + 0 + 0 + 2$   
 $= 4296015874$   
 $4296015874 / 4 = 1074003968.5$             NON INTERO, non allineato

## Operandi Immediati e Costanti

Nel caso in cui dovessimo fare un calcolo con una costante, abbiamo due possibili azioni.

1. Le costanti sono già in memoria con un indirizzo fisso. Carico dalla memoria in un registro e procedo

|                      |                    |                                                         |
|----------------------|--------------------|---------------------------------------------------------|
| <pre>b = b + 5</pre> | $\Rightarrow$      | <pre>lw x9, indirizzoCostante5(x3) add x5, x5, x9</pre> |
| Linguaggio C         | $b \rightarrow x5$ | RISC-V assembly                                         |

2. Uso l'operando addi ovvero add immediate che può sommare ad un registro un valore intero costante compreso tra -2048 e 2047. La sottrazione immediata non esiste, si usano semplicemente i negativi in addi

|                      |                    |                           |
|----------------------|--------------------|---------------------------|
| <pre>b = b + 5</pre> | $\Rightarrow$      | <pre>addi x5, x5, 5</pre> |
| Linguaggio C         | $b \rightarrow x5$ | RISC-V assembly           |

## Linguaggio Macchina

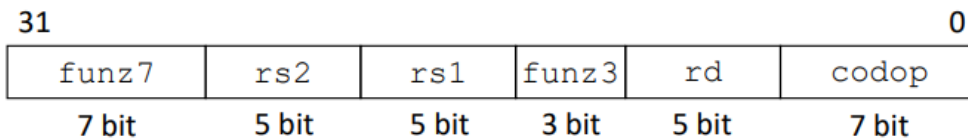
Il linguaggio macchina è esclusivamente bit, quindi 0 ed 1. Assembly è “human readable”

Ogni istruzione RISC-V richiede esattamente 32 bit per la sua rappresentazione in linguaggio macchina indipendentemente da RV32I o RV64I.

Una sequenza di istruzioni in linguaggio macchina viene chiamata codice macchina.

Abbiamo diversi formati con i quali vengono descritte informazioni e registri:

### Formato di tipo R (Registro)

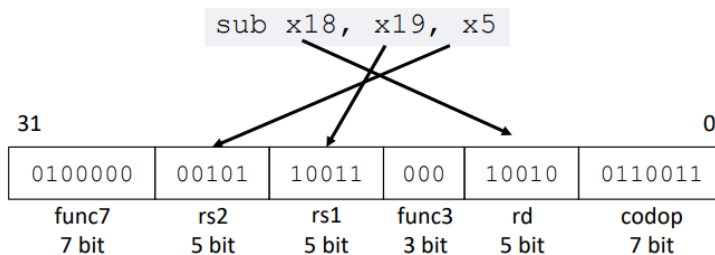


Permette di codificare le istruzioni: *add, sub, and, or, xor, ...*

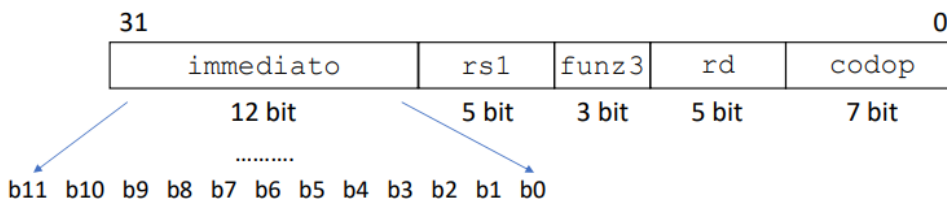
- codop                      codice operativo istruzione
- rd                         registro destinazione
- rs1                        registro che contiene il primo operando sorgente
- rs2                        registro che contiene il secondo operando sorgente
- funz3, funz7            codici operativi aggiuntivi

codop + funz3 + funz7 indicano l'istruzione rappresentata

#### • Esempio



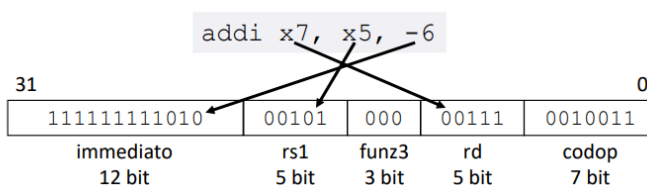
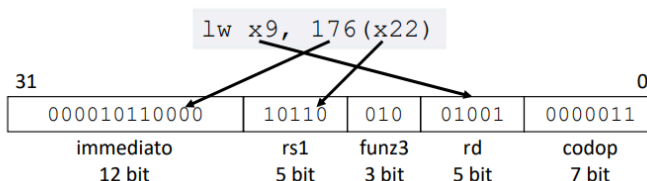
### Formato di tipo I (Immediato)



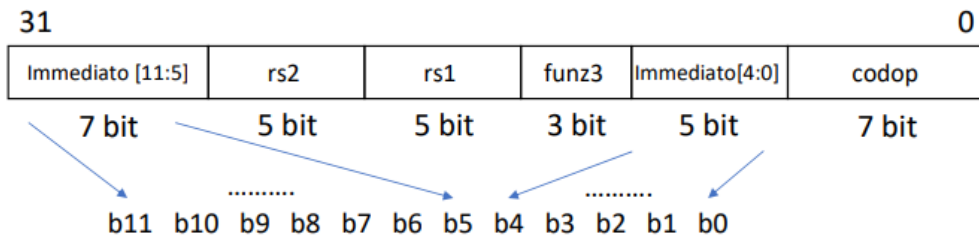
Permette di codificare le istruzioni che richiedono il caricamento dalla memoria o una costante, come: *load, addi, andi, ori, ...*

Il campo della costante (immediato) ha dimensione 12 bit per rappresentare una larga quantità di costanti. Viene codificato in complemento a due

#### • Esempi



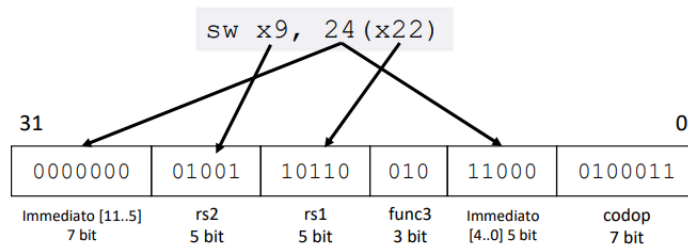
## Formato tipo S (Store)



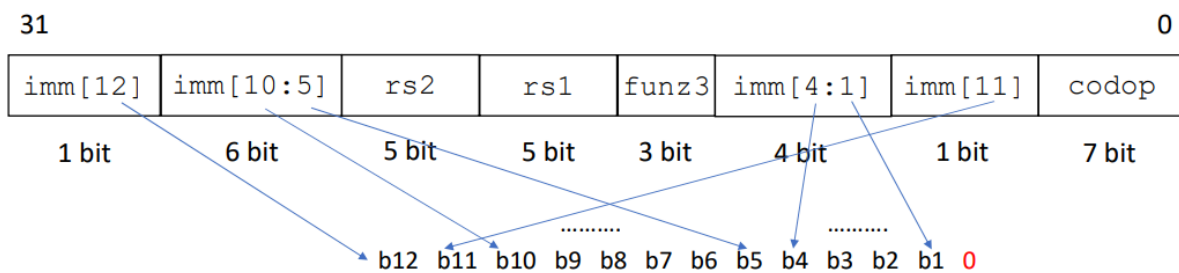
Codifica le istruzioni che richiedono il salvataggio in memoria o una costante: store, ...

Il valore immediato (offset) è diviso in 2 parti per mantenere rs2 e rs1 nella stessa posizione del tipo R.

### • Esempio



## Formato tipo SB (Branch Condizionato)



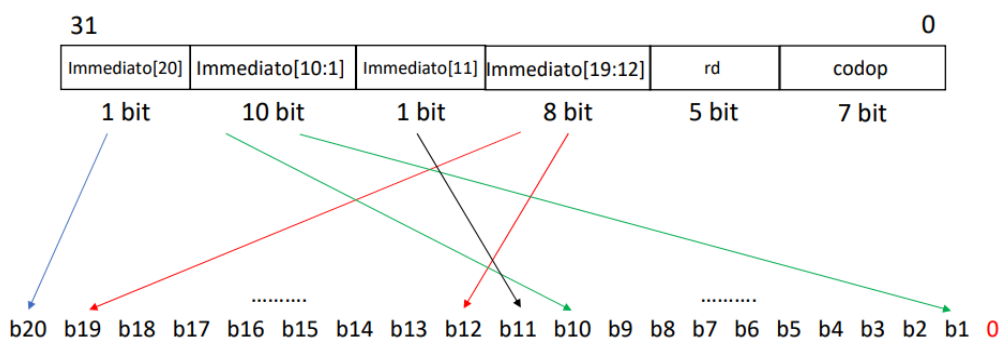
Si utilizza l'indirizzamento relativo al program counter (PC-relative addressing).

Il campo immediato di 12+1 bit contiene l'offset rispetto al valore di PC e viene espresso in complemento a due.

Per ottenere il prossimo valore di PC in caso di salto:  $PC = PC + \text{immediato}$

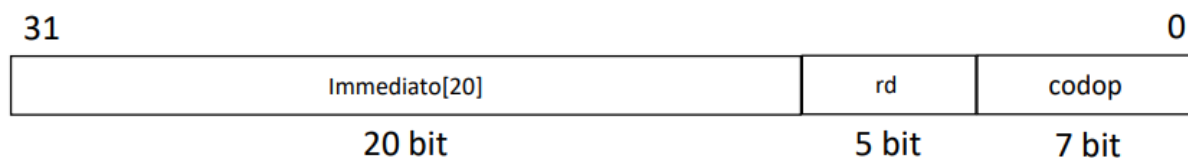
Dal momento che consideriamo le istruzioni a 32 bit, Il compilatore genererà campi immediati di istruzioni sempre multipli di 4. Questo significa che implicitamente gli ultimi due bit dell'immediato sono 0.

## Formato tipo J (Jump Type)



Anche qui assumiamo che ultimi due bit siano sempre 0

## Formato Tipo U (Upper Immediate Type)



Viene usato per gli immediati ampi. essenzialmente li scompone in due parti con 20 bit più significativi in uno e gli altri 12 in un altro

| Nome<br>(dimensione<br>del campo) | Campi                          |       |       |       |                |       | Commenti                                                          |
|-----------------------------------|--------------------------------|-------|-------|-------|----------------|-------|-------------------------------------------------------------------|
|                                   | 7 bit                          | 5 bit | 5 bit | 3 bit | 5 bit          | 7 bit |                                                                   |
| Tipo R                            | funz7                          | rs2   | rs1   | funz3 | rd             | codop | Istruzioni aritmetiche                                            |
| Tipo I                            | Immediato[11:0]                |       | rs1   | funz3 | rd             | codop | Istruzioni di caricamento dalla memoria e aritmetica con costanti |
| Tipo S                            | immed[11:5]                    | rs2   | rs1   | funz3 | immed[4:0]     | codop | Istruzioni di trasferimento alla memoria (store)                  |
| Tipo SB                           | immed[12, 10:5]                | rs2   | rs1   | funz3 | immed[4:1, 11] | codop | Istruzioni di salto condizionato                                  |
| Tipo UJ                           | immediato[20, 10:1, 11, 19:12] |       |       |       | rd             | codop | Istruzioni di salto incondizionato                                |
| Tipo U                            | immediato[31:12]               |       |       |       | rd             | codop | Formato caricamento stringhe di bit più significativi             |

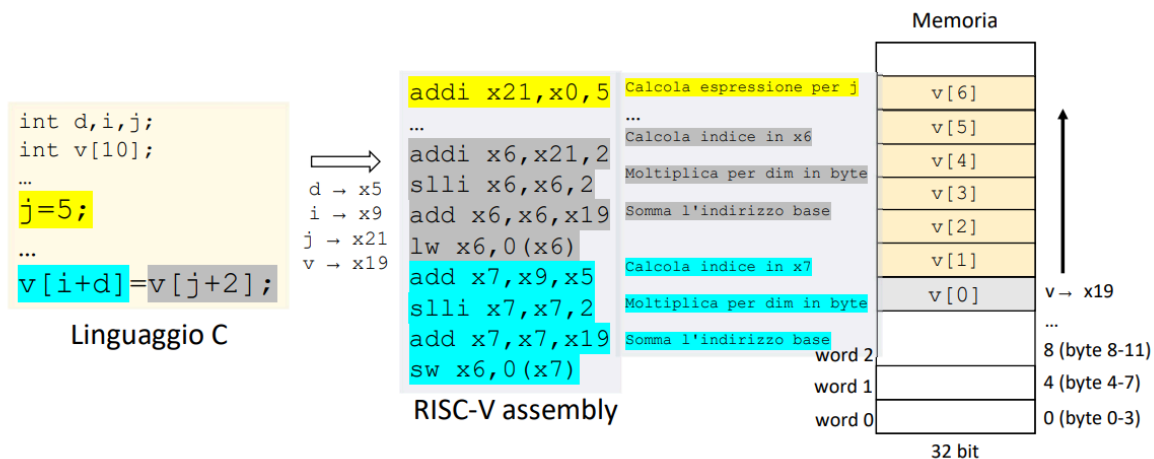
## Operazioni Logiche

Shift Logico (riempie gli extra di 0, unsigned)

- Shift Left Logical: `sll x9, x22, x19` → Formato R
- Shift Left Logical Immediate: `slli x9, x22, 5` → Formato I
- Shift Right Logical: `srl x9, x22, x19` → Formato R
- Shift Right Logical Immediate: `srlui x9, x22, x19` → Formato I

Shift Aritmetico (riempie gli extra con il segno, non unsigned)

- Shift Left Arithmetic, Immediate → non esiste, identico a sll
- Shift Right Arithmetic: `sra x9, x22, x19` → Formato R
- Shift Right Arithmetic Immediate: `srai x9, x22, 5` → Formato I



x7 contiene l'indirizzo del primo byte che rappresenta v[i+d]

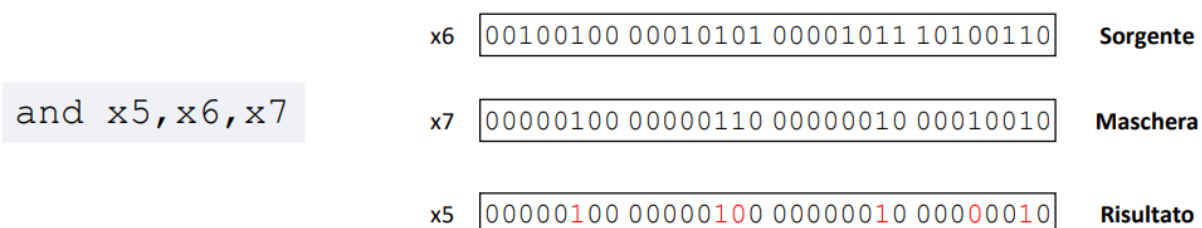
RISC-V Instruction Set

81

## Operatori Logici

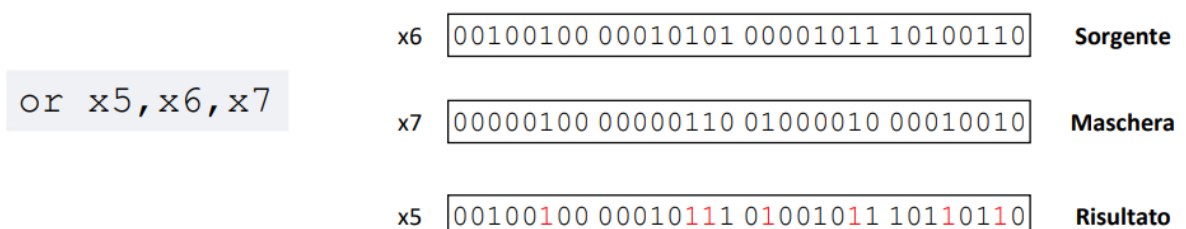
- **AND**
  - `and x9, x22, x19` → Formato R
  - `andi x9, x22, 5` → Formato I

L'AND permette di selezionare alcuni bit del primo operando indicandoli all'interno di una maschera (secondo operando)



- **OR**
  - `or x9, x22, x19` → Formato R
  - `ori x9, x22, 5` → Formato I

L'OR permette di copiare il primo operando, mettendo ad uno anche i bit che sono specificati nella maschera indicata come secondo operando



- XOR `xor x9, x22, x19` → *Formato R*  
`xori x9, x22, 5` → *Formato I*
- NOT pseudo-istruzione `not x5, x6`

### Salti Condizionati

I Salti Condizionati permettono di variare il flusso del programma (variando il valore del PC) al verificarsi di una condizione.

A = rs1

B = rs2

L1 = Operazione da eseguire se la condizione è vera

- **if A = B, do L1** `beq rs1, rs2, L1` (UGUALE) → B  
*Branch if Equal*  
Il flusso del programma continua all'istruzione L1 se il valore del registro rs1 è uguale a quello di rs2
- **if A != B, do L1** `bne rs1, rs2, L1` (DIVERSO) → B  
*Branch if Not Equal*  
Il flusso del programma continua all'istruzione L1 se il valore del registro rs1 è diverso da quello di rs2
- **if A < B, do L1** `blt rs1, rs2, L1` (MINORE) → B  
*Branch if Less Than*  
Il flusso del programma continua all'istruzione L1 se il valore del registro rs1 è minore di quello di rs2
- **if A >= B, do L1** `bge rs1, rs2, L1` (MAGGIORE =) → B  
*Branch if Greater than or Equal*  
Il flusso del programma continua all'istruzione L1 se il valore del registro rs1 è maggiore o uguale a quello di rs2
- `bltu rs1, rs2, L1` (MINORE UNSIGNED) → B
- `bgeu rs1, rs2, L1` (MAGGIORE UGUALE UNSIGNED) → B

### Istruzioni di Salto Generiche, assegnano il result ad un registro

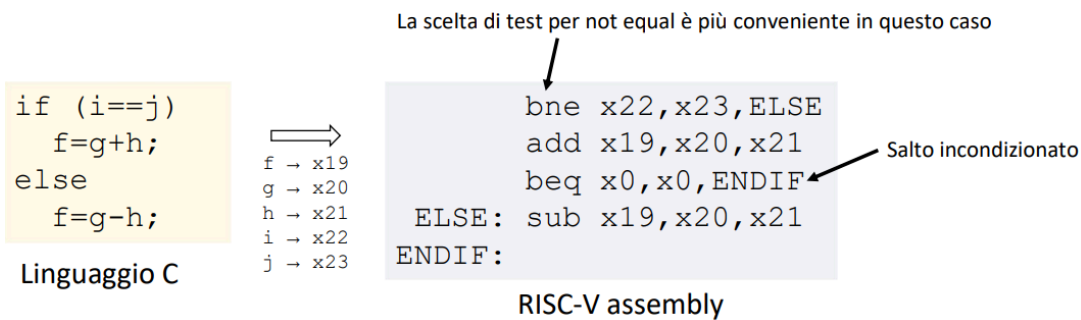
- `slt rs, rs1, rs2` se `rs1 < rs2`, rs prende valore 0 → R
- `slti rs, rs1, immediate` come il precedente ma con immediate → I
- Per confrontare >= basta invertire la condizione
- Per confrontare > basta scambiare gli operandi di `slt`
- Per confronto su <=, inverti condizione e scambio operandi

Tutte le condizioni precedenti si basano su confronti in complemento a due. Se vogliamo invece effettuare controlli unsigned, usiamo

- `sltu rs, rs1, rs2` (MINORE UNSIGNED) → R
- `sltiu rs, rs1, rs2` (MINORE IMMEDIATE UNSIGNED) → I

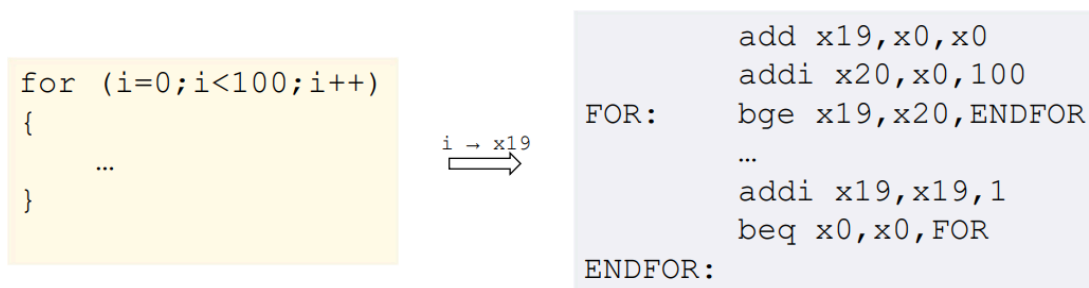
## Salti condizionati: costrutto if-then-else

- Attraverso le istruzioni `beq` e `bne` è possibile tradurre in assembly il costrutto `if` dei linguaggi di programmazione ad alto livello

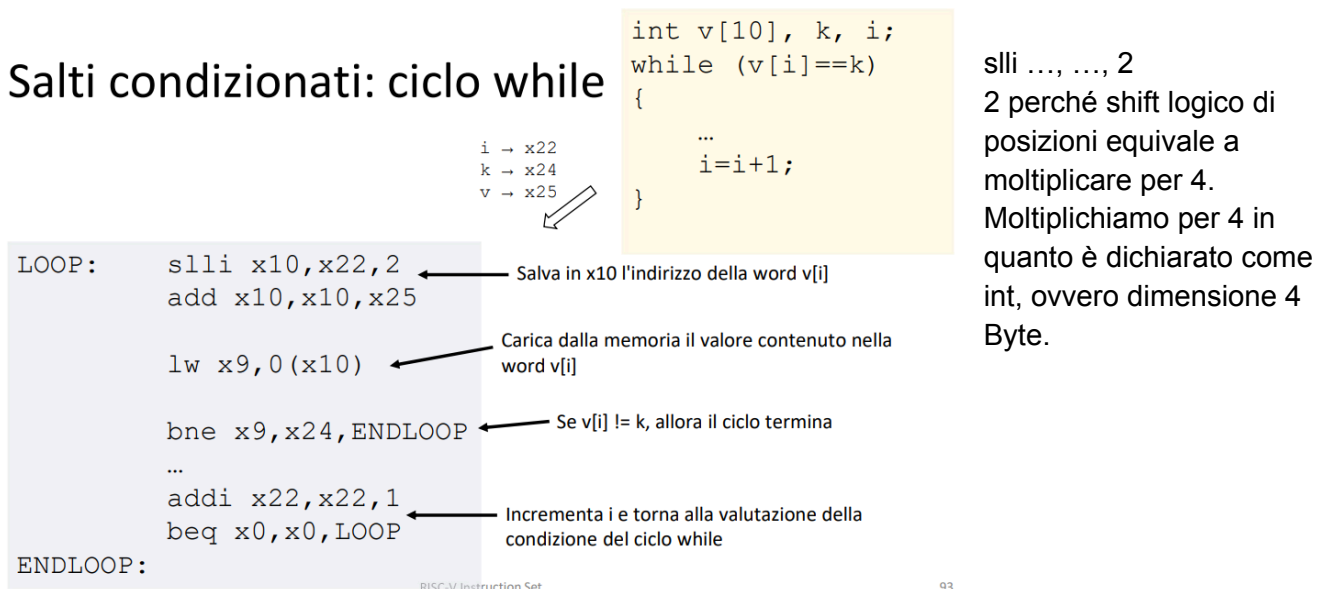


## Salti condizionati: ciclo for

- Una possibile implementazione



## Salti condizionati: ciclo while



## PSEUDOISTRUZIONI

Le pseudo istruzioni esistono solo nel linguaggio assembly ma non in quello macchina. Il compilatore traduce le pseudo istruzioni in istruzioni più native.

- mv x5, x6 → addi x5, x6, 0
- not x5, x6 → nxori x5, x6, -1

### Istruzioni Aritmetiche: Moltiplicazione

Quanto effettuiamo una moltiplicazione tra due int di dimensione massima  $2^{32}$ , possiamo ottenere un valore fino a  $2^{64}$ . Per questo motivo viene usato un secondo registro interno nascosto all'utente per svolgere il calcolo. In base all'istruzione possiamo ottenere parte del risultato del calcolo.

```
a = b * c →  
mul a, b, c Ottieni la parte MENO significativa [0:32]  
mulh a, b, c Ottieni la parte PIÙ significativa [32:64]
```

Per le versioni unsigned, come sempre, aggiungi u alla fine.

### Istruzioni Aritmetiche: Divisione e Resto

```
a = b / c → div a, b, c Divisione Intera  
a = b % c → rem a, b, c Resto Intero della divisione
```

Per le versioni unsigned, come sempre, aggiungi u alla fine.

## PROCEDURE

Porzioni di codice associate ad un nome che possono essere invocate più volte e che eseguono un compito specifico avendo come input una lista di parametri e come output un valore di ritorno.

Nell'uso di una procedure abbiamo

#### Chiamante

- Mette i parametri in un luogo accessibile alla procedura
- Trasferisce il controllo alla procedura

#### Chiamato

- Acquisisce le risorse necessarie all'esecuzione
- Esegue il compito richiesto
- Mette il risultato in un luogo accessibile al chiamante
- Restituisce il controllo al punto di origine

### Modifica del flusso di programma: Invocazione

Jump-and-link jal x1, IndirizzoProcedura → Type: J

- Salta all'indirizzo (offset) con etichetta IndirizzoProcedura
- Memorizza l'indirizzo dell'istruzione successiva (PC+4) nel registro X1 (return address, **ra**)
- è un salto RELATIVO alla posizione attuale del PC

Abbiamo anche la pseudo-istruzione senza x1: j IndirizzoProcedura

Jump-and-link-register jalr rd, offset(rs1) → Type: I

- Salta all'indirizzo indicato + offset
- Memorizza l'indirizzo dell'istruzione successiva (PC+4) nel registro X1 (return address, **ra**)
- è un salto ASSOLUTO ad un indirizzo ed offset

Abbiamo anche la pseudo-istruzione senza x1: jr ra

ra viene rappresentato in complemento a 2 nel formato I

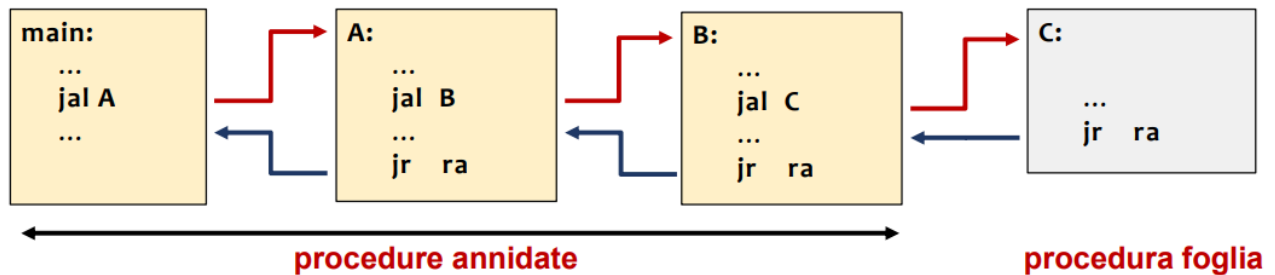
Quando chiamo una funzione, poi alla fine chiamo jr ra per tornare al chiamante (o solo ret)

## Sovrascrittura Registri, Procedure Annidate

Sia nel caso di grandi calcoli che di procedure annidate, potrebbe capitare di dover sovrascrivere uno dei registri da usare.

- **Soluzione:**

- La procedura chiamata, deve **salvare in memoria** il valore di `x1` **prima** di chiamare la procedura annidata con l'istruzione `jal`



## Lo Stack

Lo Stack è una struttura di dati dinamica con accesso Last In First Out (LIFO)

- **PUSH:** aggiunge un elemento "in cima allo stack"
- **POP:** rimuove un elemento dalla "cima dello stack"

La cima dello stack è identificata dallo Stack Pointer (SP)

Convenzioni:

- grow-down: lo stack cresce da indirizzi di memoria alti verso bassi (-4, -8, ...)
- last-full: lo stack pointer contiene indirizzo dell'ultima cella di memoria
- **il valore di SP è salvato nel registro x2**
- I registri x10-x17 (a0-a7), x5-x7, x28-x31 (t0, t6)
  - possono essere modificati dal chiamato senza nessun ripristino
  - se il chiamante deve continuare a usare quei dati, li deve salvare in memoria
- I registri x1 (ra), x2, (sp), x3 (gp), x4 (tp), x8(fp/s0), x9, x18-x27 (s1-s11)
  - se modificati dal chiamato, devono essere salvati e ripristinati prima del return
  - il chiamante NON è tenuto a salvarli

### PUSH:

- Decrementa SP                      `addi sp, sp, -4`                      Inizializzaz. frame 4-Byte
- Scrive in M[SP]                      `sw x20, 0(sp)`

### POP:

- Legge da M[SP]                      `lw x20, 0(sp)`
- Incrementa SP                      `addi sp, sp, 4`                      Terminazione frame 4-Byte

Il 4 chiaramente è nel caso di RV32 in quanto gli indirizzi sono da 32 Bit (4 Byte). Se fossimo in RV64, sarebbe 8.

Ogni procedura chiamata, genera quindi un nuovo elemento nello stack che viene rimosso a fine esecuzione. In cima allo stack ci sarà il frame attualmente in esecuzione. Il frame non è un singolo dato ma il campione in uso dalla procedura in corso.

Abbiamo due dati rilevanti rispetto al frame in esecuzione:

- Base:                      Frame Pointer                      registro x8,    fp
- Cima:                      Stack Pointer                      registro x2,    sp

Il fp resta fermo nell'esecuzione di una procedura, permettendoci di accedere comodamente ai suoi valori

## Le fasi di una Invocazione di Procedura

1. Pre-Chiamata
2. Invocazione della Procedura
  - a. Prologo del chiamato
  - b. Corpo della procedura
  - c. Epilogo lato chiamato
  - d. Ritorno al Chiamante
3. Post-Chiamata

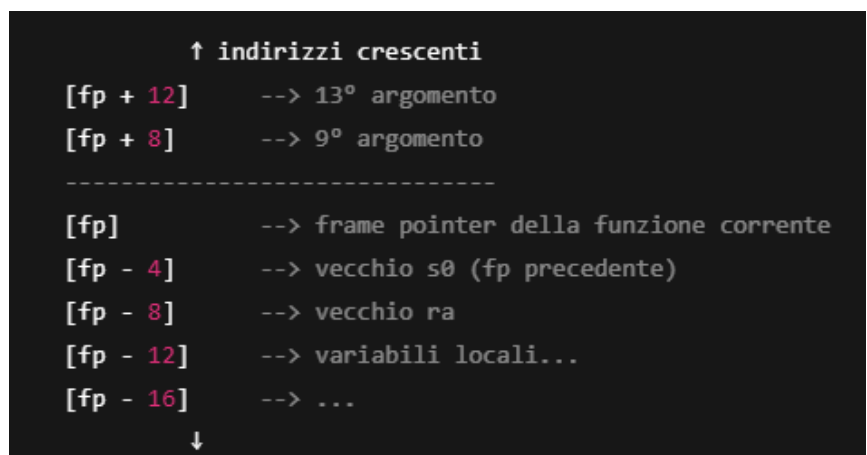
Dove 1,2,3 sono del chiamate. a,b,c,d sono del chiamato

1. Pre-Chiamata del Chiamante
  - a. Eventuale salvataggio registri da preservare
  - b. Preparazione argomenti funzione
    - i. I primi 8 valori vengono posti in x10-x17 (a0-a7)
    - ii. Gli eventuali ulteriori vanno nello stack così da essere in cima al frame
2. Invocazione della procedura
  - a. Istruzione `jal NOME_PROCEDURA`
3. Prologo lato Chiamato
  - a. Eventuale allocazione del cal-frame sullo stack
    - i. Salva x1 (ra) se la procedura non è una foglia
    - ii. Salvataggio x8 (fp) se usato nella procedura
    - iii. Salvataggio di x9 e x18-x27 (s1-s11) se utilizzati (il chiamante si aspetta di trovarli lì al return)
    - iv. Salvataggio argomenti x10-x17 (a0, a7) se utilizzati in seguito ad altre chiamate di procedure
4. Corpo della procedura
  - a. Istruzioni che implementano la procedura
5. Epilogo lato Chiamato
  - a. Se deve avere un return, mette il valore in x10 e x11 (a0-a1)
  - b. Ripristina i registri che aveva salvato
  - c. Incremento sp dello stesso valore per il quale lo avevo decrementato prima
6. Ritorno al Chiamante
  - a. Istruzione `jr ra`
7. Post-Chiamata lato chiamante
  - a. Eventuale uso del risultato della funziona
  - b. Ripristino dei vecchi valori salvati

Semplificazione:

Se la procedura è una foglia

- non è necessario salvare e ripristinare ra
- non è necessario salvare e ripristinare a0-a7, t0-t6



## Operandi Immediati Ampì

Se dobbiamo caricare una costante lunga 32 Bit in un registro, come facciamo ?

Il dato viene salvato nel formato U e si compone in questo modo:

dato da salvare = 0x12345678

1. lui x5, 0x12345      Carica i 20 bit più significativi della costante
2. ori x4, x5, 0x678    Imposto i 12 bit meno significativi rimasti

### Reindirizzamento con Offset Ampì

Se dobbiamo fare un reindirizzamento lontano, RISC-V introduce un'istruzione che permette salti fino a  $2^{32}$

- auipc rd, offset      Add Upper Immediate PC      →      Tipo U

Ha come effetto l'inserimento nel registro rd l'indirizzo di PC + (offset << 12) (left logical shift di 12)

# Alcune delle pseudo-istruzioni presenti nel RISC-V

|                     |                         |                                 |
|---------------------|-------------------------|---------------------------------|
| nop                 | addi x0, x0, 0          | No operation                    |
| li rd, immediate    | <i>Myriad sequences</i> | Load immediate                  |
| mv rd, rs           | addi rd, rs, 0          | Copy register                   |
| not rd, rs          | xori rd, rs, -1         | One's complement                |
| neg rd, rs          | sub rd, x0, rs          | Two's complement                |
| negw rd, rs         | subw rd, x0, rs         | Two's complement word           |
| sext.w rd, rs       | addiw rd, rs, 0         | Sign extend word                |
| seqz rd, rs         | sltiu rd, rs, 1         | Set if = zero                   |
| snez rd, rs         | sltu rd, x0, rs         | Set if $\neq$ zero              |
| sltz rd, rs         | slt rd, rs, x0          | Set if < zero                   |
| sgtz rd, rs         | slt rd, x0, rs          | Set if > zero                   |
| fmv.s rd, rs        | fsgnj.s rd, rs, rs      | Copy single-precision register  |
| fabs.s rd, rs       | fsgnjx.s rd, rs, rs     | Single-precision absolute value |
| fneg.s rd, rs       | fsgnjn.s rd, rs, rs     | Single-precision negate         |
| fmv.d rd, rs        | fsgnj.d rd, rs, rs      | Copy double-precision register  |
| fabs.d rd, rs       | fsgnjx.d rd, rs, rs     | Double-precision absolute value |
| fneg.d rd, rs       | fsgnjn.d rd, rs, rs     | Double-precision negate         |
| beqz rs, offset     | beq rs, x0, offset      | Branch if = zero                |
| bnez rs, offset     | bne rs, x0, offset      | Branch if $\neq$ zero           |
| blez rs, offset     | bge x0, rs, offset      | Branch if $\leq$ zero           |
| bgez rs, offset     | bge rs, x0, offset      | Branch if $\geq$ zero           |
| bltz rs, offset     | blt rs, x0, offset      | Branch if < zero                |
| bgtz rs, offset     | blt x0, rs, offset      | Branch if > zero                |
| bgt rs, rt, offset  | blt rt, rs, offset      | Branch if >                     |
| ble rs, rt, offset  | bge rt, rs, offset      | Branch if $\leq$                |
| bgtu rs, rt, offset | bltu rt, rs, offset     | Branch if >, unsigned           |
| bleu rs, rt, offset | bgeu rt, rs, offset     | Branch if $\leq$ , unsigned     |

| pseudoinstruction | Base Instruction                                                  | Meaning                           |
|-------------------|-------------------------------------------------------------------|-----------------------------------|
| j offset          | jal x0, offset                                                    | Jump                              |
| jal offset        | jal x1, offset                                                    | Jump and link                     |
| jr rs             | jalr x0, 0(rs)                                                    | Jump register                     |
| jalr rs           | jalr x1, 0(rs)                                                    | Jump and link register            |
| ret               | jalr x0, 0(x1)                                                    | Return from subroutine            |
| call offset       | auipc x1, offset[31:12] + offset[11]<br>jalr x1, offset[11:0](x1) | Call far-away subroutine          |
| tail offset       | auipc x6, offset[31:12] + offset[11]<br>jalr x0, offset[11:0](x6) | Tail call far-away subroutine     |
| fence             | fence iorw, iorw                                                  | Fence on all memory and I/O       |
| rdinstret[h] rd   | csrrs rd, instret[h], x0                                          | Read instructions-retired counter |
| rdcycle[h] rd     | csrrs rd, cycle[h], x0                                            | Read cycle counter                |
| rdtime[h] rd      | csrrs rd, time[h], x0                                             | Read real-time clock              |
| csrr rd, csr      | csrrs rd, csr, x0                                                 | Read CSR                          |
| csrw csr, rs      | csrrw x0, csr, rs                                                 | Write CSR                         |
| csrs csr, rs      | csrrs x0, csr, rs                                                 | Set bits in CSR                   |
| csrc csr, rs      | csrrc x0, csr, rs                                                 | Clear bits in CSR                 |
| csrwi csr, imm    | csrrwi x0, csr, imm                                               | Write CSR, immediate              |
| csrsi csr, imm    | csrrsi x0, csr, imm                                               | Set bits in CSR, immediate        |
| csrci csr, imm    | csrrci x0, csr, imm                                               | Clear bits in CSR, immediate      |
| frcsr rd          | csrrs rd, fcsr, x0                                                | Read FP control/status register   |
| fscsr rd, rs      | csrrw rd, fcsr, rs                                                | Swap FP control/status register   |
| fscsr rs          | csrrw x0, fcsr, rs                                                | Write FP control/status register  |
| frfm rd           | csrrs rd, frm, x0                                                 | Read FP rounding mode             |
| fsrm rd, rs       | csrrw rd, frm, rs                                                 | Swap FP rounding mode             |
| fsrm rs           | csrrw x0, frm, rs                                                 | Write FP rounding mode            |
| frflags rd        | csrrs rd, fflags, x0                                              | Read FP exception flags           |
| fsflags rd, rs    | csrrw rd, fflags, rs                                              | Swap FP exception flags           |
| fsflags rs        | csrrw x0, fflags, rs                                              | Write FP exception flags          |

## Il livello del Linguaggio: Assembly

1. (Livello 4: Sorgente C)                      Programma C  
    → [Compilatore] →
2. (Livello 3: RISC-V Assembly)              Programma in linguaggio Assembler  
    → [Assembler] →
3. (Livello 1: ISA: binario)  
    Oggetto:  
        modulo in linguaggio macchina +  
        funzioni di libreria  
    → (Linker) →
4. Eseguitibile: programma in linguaggio macchina  
    → (Loader) →
5. Memoria ed esecuzione

Quando si parla di Linguaggio Assembly si intende un linguaggio le cui istruzioni sono ottenute dalle istruzioni SIA sostituendo i codici binari con codici mnemonici; il linguaggio assembly è quindi molto vicino al linguaggio macchina: c'è sostanzialmente una corrispondenza uno-uno tra le istruzioni SIA e le istruzioni del linguaggio assembly. Ci sono inoltre alcune semplificazioni per gli sviluppatori come

- etichette
- primitive di allocazione di memoria
- costanti
- macro, pseudo-istruzioni

### Macro

Esistono anche le macro, a differenza delle procedure, vengono sostituite in fase di assemblaggio. non c'è quindi una chiamata a funzione con return ecc ma solo una sostituzione della porzione di testo nella macro con il testo della macro. Funzionano come ogni macro insomma

### Assemblatore

Ci sono diverse fasi che compongono l'assemblaggio del file oggetto.

1. Individuazione di tutti i nomi (etichette) che compaiono
  - a. Creazione di una Symbol Table che contiene le etichette con la loro posizione relativa all'interno del programma
2. Traduzione dei codici mnemonici delle istruzioni, degli operandi e delle etichette mediante la consultazione della Symbol Table

All'interno della Symbol Table troviamo **ILC** ovvero il contatore che tiene traccia degli indirizzi di memoria delle istruzioni. Sostituisce inoltre gli immediati con il loro valore binario

| Label     | Opcode | Operands | Comments                    | Length | ILC |
|-----------|--------|----------|-----------------------------|--------|-----|
| MARIA:    | MOV    | EAX, I   | EAX = I                     | 5      | 100 |
|           | MOV    | EBX, J   | EBX = J                     | 6      | 105 |
| ROBERTA:  | MOV    | ECX, K   | ECX = K                     | 6      | 111 |
|           | IMUL   | EAX, EAX | EAX = I * I                 | 2      | 117 |
|           | IMUL   | EBX, EBX | EBX = J * J                 | 3      | 119 |
|           | IMUL   | ECX, ECX | ECX = K * K                 | 3      | 122 |
| MARILYN:  | ADD    | EAX, EBX | EAX = I * I + J * J         | 2      | 125 |
|           | ADD    | EAX, ECX | EAX = I * I + J * J + K * K | 2      | 127 |
| STEPHANY: | JMP    | DONE     | branch to DONE              | 5      | 129 |

## Linking e Loading

- **Programmi:** insieme di procedure (moduli) tradotti separatamente dall'assemblatore. Ogni modulo oggetto ha il suo spazio di indirizzamento separato.
- **Linker:** è un programma che collega i moduli oggetto in modo da formare un unico modulo eseguibile

Ogni modulo ha un proprio spazio di indirizzamento che parte sempre da 0 (per avere un offset da inizio file)

Il linker fonde gli spazi di indirizzamento dei moduli oggetto in uno spazio lineare unico

1. Costruisce una tabella di tutti i moduli oggetto e le loro lunghezze. Così da avere un riferimento ed una dimensione dei salti
2. Assegna un indirizzo di inizio ad ogni modulo oggetto
3. Trova tutte le istruzioni che accedono alla memoria e aggiunge a ciascun indirizzo una **relocation constant** corrispondente all'indirizzo di partenza del suo modulo
4. Trova tutte le istruzioni che fanno riferimento ad altri moduli e le aggiorna con l'indirizzo corretto

## Struttura di un file Oggetto

1. Identification
  - a. Nome del modulo
  - b. Lunghezza delle varie sezioni
  - c. Data di assemblaggio
2. Entry Point Table
  - a. Elenco dei simboli esportati, ovvero quelli che gli altri moduli possono usare (funzioni, variabili pubbliche, ...)
  - b. Indirizzi relativi nel modulo
3. External Reference Table (usato dal linker per sapere cosa risolvere esternamente)
  - a. Elenco dei simboli usati ma non definiti nel modulo
  - b. Indica dove sono usati (quali istruzioni li richiamano)
4. Machine Instructions And Constants
  - a. Codice macchina vero e proprio
  - b. Dati e costanti
5. Relocation Dictionary
  - a. Elenco degli indirizzi da rilocare
  - b. Ogni voce ha un flag di rilocazione (1 = da sistemare, il linker non sa ancora quali parti del programma sono funzioni e quali costanti, quindi annota come da sistemare)
6. End Module
  - a. Possibili messaggi di errore
  - b. Indirizzo di inizio esecuzione

## Loader

Una volta creato l'eseguibile mediante il linker, viene salvato in memoria fisica. Quando deve essere eseguito, il sistema operativo lo carica in memoria centrale e ne avvia l'esecuzione.

Il loader è un programma del sistema operativo e si occupa di:

1. Leggere l'intestazione per determinare la dimensione del programma e dei dati
2. Riservare uno spazio in memoria sufficiente per contenerli
3. Copiare programma e dati nello spazio riservato
4. Copiare nello stack i parametri passati al main
5. Inizializzare tutti i registri e lo stack pointer (ma anche gli altri del modello di memoria)
6. Saltare ad una procedura che copia i parametri dallo stack ai registri e che poi invoca il main

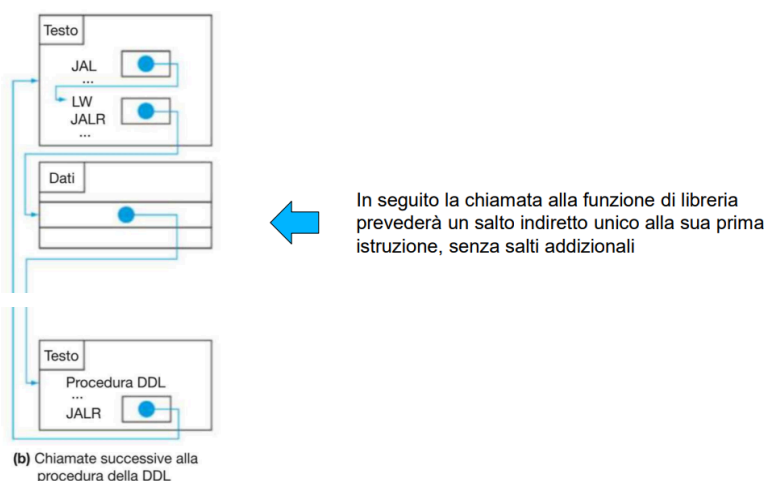
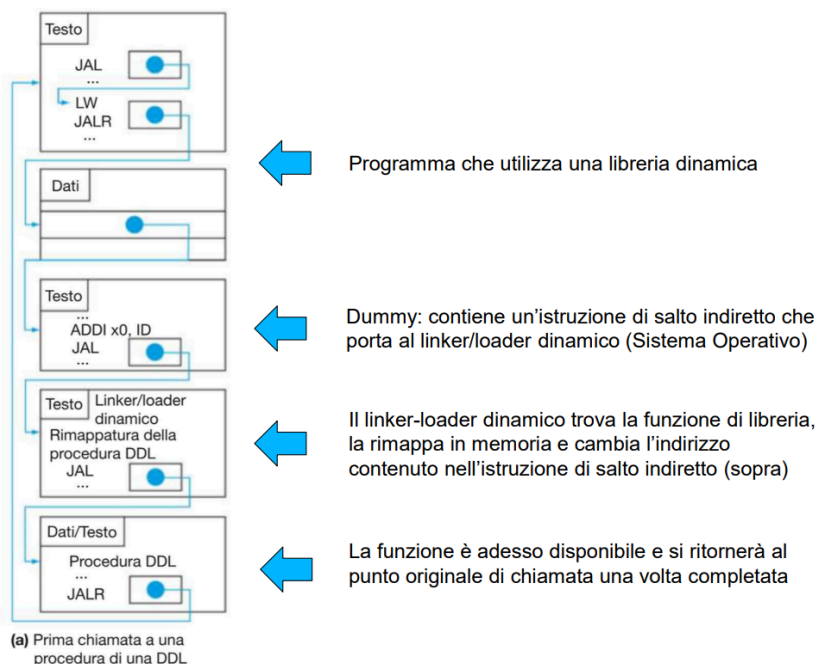
## Binding e Rilocalizzazione Dinamica

Collegamento Statico:

- Le funzioni di libreria diventano parte del codice sorgente
- Se la libreria viene aggiornata, il codice continua comunque ad usare quelle vecchie
- Le librerie possono essere enormi e rendere un codice piccolo, molto pesante
- Esecuzione diretta

Collegamento Dinamico

- DLL, Dynamically Linked Libraries
- Le funzioni di libreria non vengono collegate e caricate finché non si inizia l'esecuzione del programma
- DLL con collegamento lazy: ogni procedura viene caricata solo dopo la sua prima chiamata



# Il livello logico digitale: Circuiti Logici digitali di Base

Si tratta del livello hardware con il quale vengono eseguite istruzioni e programmi.

Si compone di:

- Porte
- Registri
- Memoria

Sotto questo livello ci sono solamente i transistor.

**CONVENZIONI DI RAPPRESENTAZIONE:** Due linee non “interagiscono” a meno che non ci sia un punto esplicito sopra.

## Circuiti Digitali

Sono gli elementi di base con cui si costruiscono i calcolatori.

Sono dispositivi che utilizzano solo due valori logici:

- 0        segnale tra 0 e 1 volt
- 1        segnale tra 2 e 5 volt

Un circuito digitale trasforma segnali (binari) di ingresso nei segnali (binari) di uscita

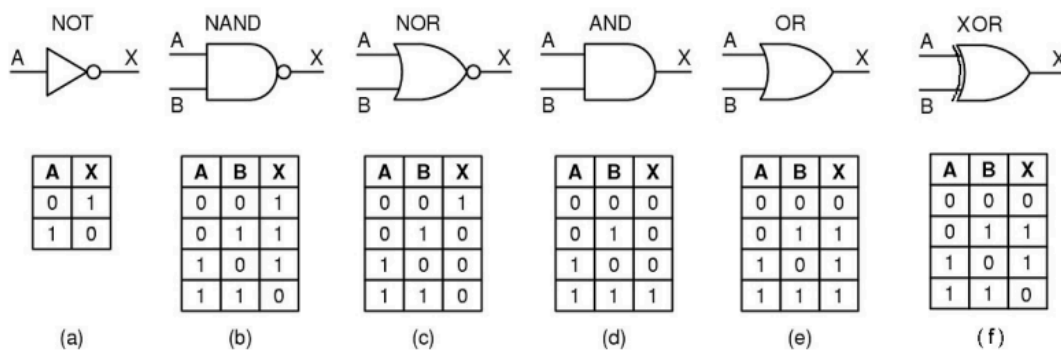
- **Circuiti COMBINATORI:** l'output della funzione dipende solamente dall'input. Non hanno stati di memoria che “ricordano” passaggi precedenti
- **Circuiti SEQUENZIALI:** l'output della funzione dipende anche dallo stato di una procedura precedente. Quindi “ricorda” i passaggi precedenti ed ha una memoria

## Porte Logiche

Sono gli elementi primitivi dei circuiti digitali e calcolano alcune funzioni di questi segnali a due valori.

Questi dispositivi si basano sul fatto che si può far funzionare un transistor come un interruttore binario molto veloce.

## Porte logiche



## Combinatorio: Decoder

Prende un numero di  $n$  bit come ingresso e lo usa per selezionare (mettere ad 1, asserire) una delle  $2^n$  linee di uscita.

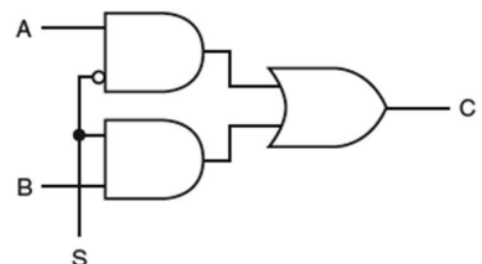
## Combinatorio: Multiplexer - Selettori

Il multiplexer permette di mandare in output uno dei due valori in ingresso in base ad un selettore.

Abbiamo 3 input: A, B, S (selettore)

Se

- $S = 0$         →        Output: A
- $S = 1$         →        Output: B



In generale:  $2^n$  ingressi, 1 uscita,  $n$  ingressi di controllo. è composto da and e un or finale.

## Circuiti numerici: Addizioni

Il blocco addizione riceve in input:

- Due bit (cifre in base 2) da sommare:
- Un bit (cifra in base 2) di riporto:

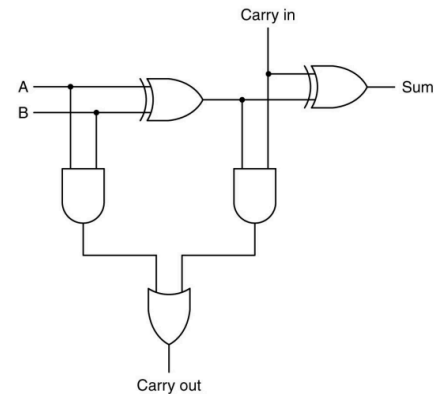
Restituisce:

- Un bit (cifra in base 2) che rappresenta il risultato
- Un bit (cifra in base 2) che rappresenta il riporto

a, b  
CarryIn

Sum  
CarryOut

## Addizionatori con XOR



## RISC-V: ALU

### Arithmetic Logic Unit

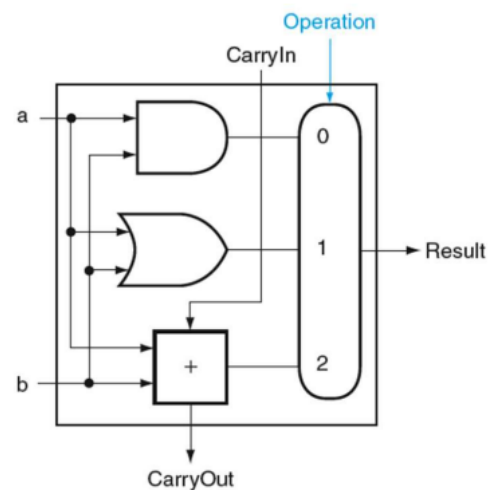
Riceve in input: a, b, CarryIn, Operation, Ainvert, Binvert, Less

Restituisce: Result, CarryOut

Può eseguire:

- a AND b (bit a bit)
- a OR b (bit a bit)
- a + b (somma su 64 cifre)

- Una ALU per operandi a, b ad n operandi si ottiene utilizzando n ALU ad 1 bit.
- Ogni ALU esegue l'operazione su una coppia di bit nella stessa posizione (ordinati)
- Per eseguire la somma, il CarryOut di un'operazione diventa il CarryIn della successiva
- Il CarryIn del bit meno significativo può essere usato come segnale di incremento



## Continuiamo ad espandere l'ALU: Sottrazioni

Vengono aggiunte due componenti "Inverter" subito dopo gli input a, b. Mediante queste componenti possiamo calcolare il complementare dell'input (quindi il suo inverso + 1, il negativo).

Possiamo usare inverter di a ed inverter di b per ottenere NOR: not (A or B)

Viene usato il CarryIn = 1 per il bit meno significativo

Multiplexer

## ALU per slt (SET ON LESS THAN)

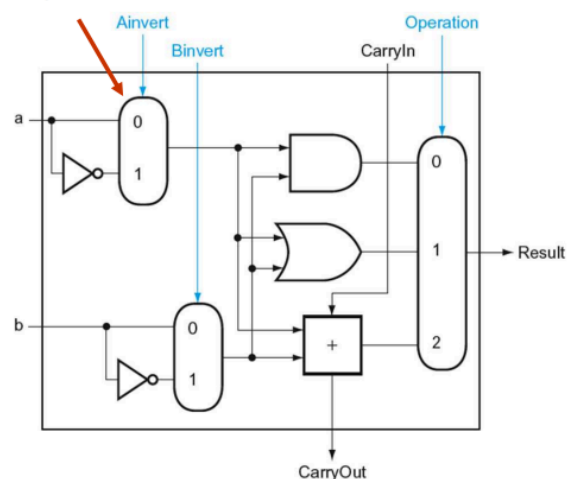
L'istruzione SET ON LESS THAN (slt) restituisce

1 se  $rs1 < rs2$

Output effettivo: tutti zero eccetto l'ultimo numero che dipende dal confronto.

Se voglio verificare  $rs1 < rs2$ , posso farlo come  $(rs1 - rs2) < 0$  ? se sì,  $rs1 < rs2$ .

Il controllo  $< 0$  è semplice, basta verificare il bit + significativo



## OverFlow detection

Dato che nella somma (e sottrazione) si possono ottenere overflow, si aggiunge un meccanismo di controllo dell'overflow.

Abbiamo overflow se:

- I due operandi hanno lo stesso segno ma l'output ne ha uno opposto
- Quando il CarryIn e CarryOut del bit più significativo sono discordi

## ALU per beq (Branch if Equal)

Salto condizionato se i due valori sono uguali.

Viene realizzato eseguendo  $(a - b)$  e controllando se il risultato è  $== 0$

L'ALU restituisce 1 quando tutti i bit di  $(a - b) == 0$

## Rappresentazione ALU

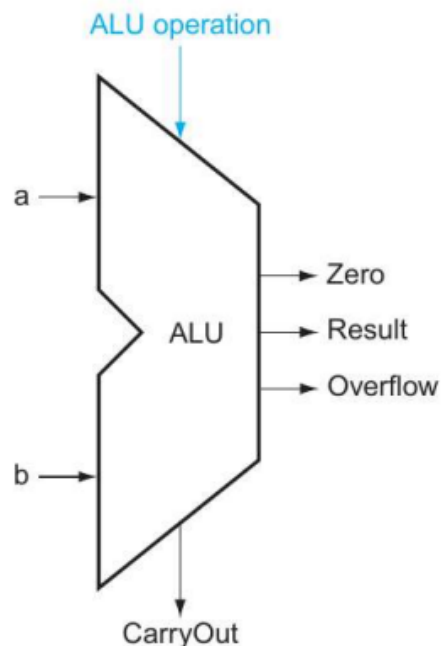
Il simbolo qui di seguito è usato per indicare anche un addizionatore, è buona norma scriverci sopra di quale componente si tratta.

## Controllo operazioni dell'ALU

Per il controllo delle operazioni possiamo usare una serie di combinazioni:

- 1 bit per l'ingresso Ainvert
- 1 bit per l'ingresso Bnegate
- 2 bit per gli ingressi Operation

Possiamo quindi ottenere le seguenti combinazioni:



| Ainvert | Bnegate | Operation |
|---------|---------|-----------|
| 0       | 0       | 00        |
| 0       | 0       | 01        |
| 0       | 0       | 10        |
| 0       | 1       | 10        |
| 0       | 1       | 11        |
| 1       | 1       | 00        |

| ALU control lines | Function      |
|-------------------|---------------|
| 0000              | AND           |
| 0001              | OR            |
| 0010              | add           |
| 0110              | subtract      |
| 0111              | set less than |
| 1100              | NOR           |

## Circuiti Sequenziali

Sono circuiti dove l'output non dipende solamente dall'input ma anche da uno stato dato da un'operazione precedente (storico).

Dispositivo di memorizzazione per 1 bit: **latch** a 1 bit

- due ingressi i, b
- un uscita o
- mantiene uno stato interno s

Se  $b = 1$  (store), input  $\rightarrow$  stato  $\rightarrow$  output (viene impostato lo stato e restituito in output)  
Se  $b = 0$  (hold), ignora  $\gg$  stato  $\rightarrow$  output (viene usato lo stato attuale)

## Clock

Un circuito che emette una serie di impulsi con una specifica larghezza e intermittenza

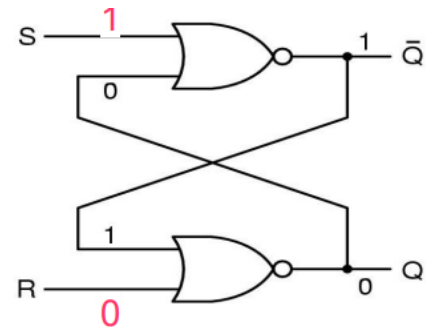
**Tempo di ciclo clock:** intervallo fra i fronti corrispondenti di due impulsi consecutivi. Il tempo si misura in sottomultipli di secondo.

**Frequenza di clock:** specifica il numero di periodi di clock per unità di tempo (per secondo).

- 1 ms (millisecond) =  $1 \cdot 10^{-3}$  sec | Frequenza = 1 KHz (KiloHertz)
- 1  $\mu$ s (microsecondo) =  $1 \cdot 10^{-6}$  sec | Frequenza = 1 MHz (MegaHertz)
- 1 ns (nanosecondo) =  $1 \cdot 10^{-9}$  sec | Frequenza = 1 GHz (GigaHertz)

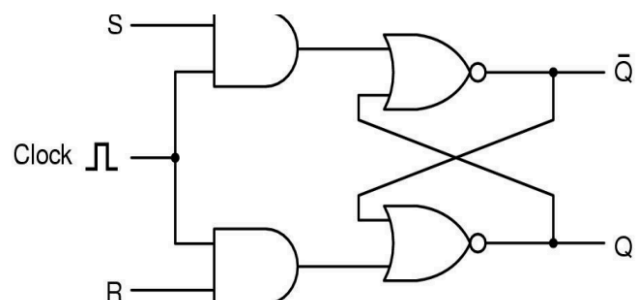
**Latch SR:** non sincronizzato con un clock, si spacca se  $R = S = 1$ , due input e un output:

- R Reset
- S Set



**Latch SR sincronizzato:** sincronizzato con il clock, tre input:

- R Reset
- S Set
- Clock Gestisce il trigger del cambio di stato in memoria, propaga solo se = 1

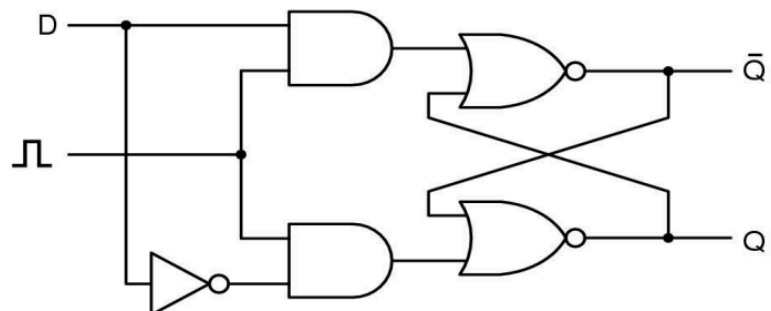


**Latch D sincronizzato:** sincronizzato con il clock, due input

- D Data
- Clock Gestisce il trigger del cambio di stato in memoria, propaga solo se = 1

Questo latch evita l'ambiguità di  $R=S=1$ .

- $D = 1, C = 1 \rightarrow$  Stato 1
- $D = 0, C = 1 \rightarrow$  Stato 0
- $D = \text{Any}, C = 0 \rightarrow$  Stato



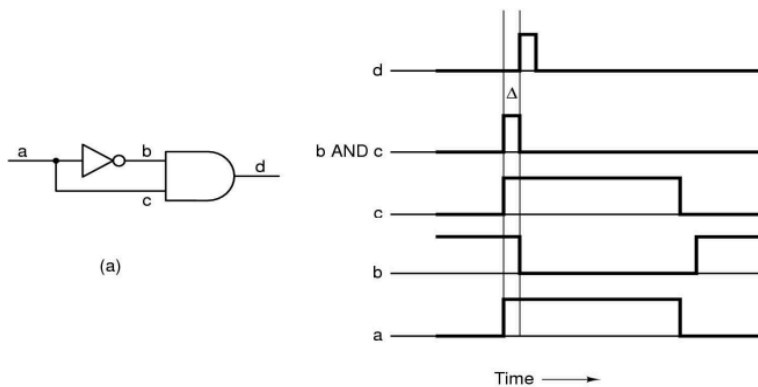
**Commutazione a livello:** la commutazione avviene sul livello alto (o basso) del clock. D-Latch.

C'è il problema che un segnale di ritorno "sporco" potrebbe interferire con lo stato salvato.

**Commutazione sul fronte:** la commutazione avviene sul fronte di salita (o discesa) del clock, non sull'intero stato alto. Questo significa che è un'azione istantanea ed il ritorno di un segnale "sporco" non fa in tempo ad arrivare per essere registrato.

## Flip-Flop di tipo D

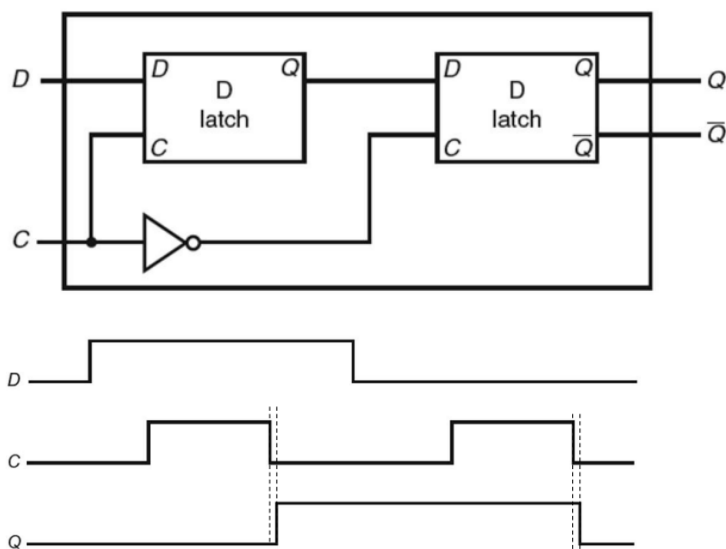
### - Commutazione sul Fronte in Salita



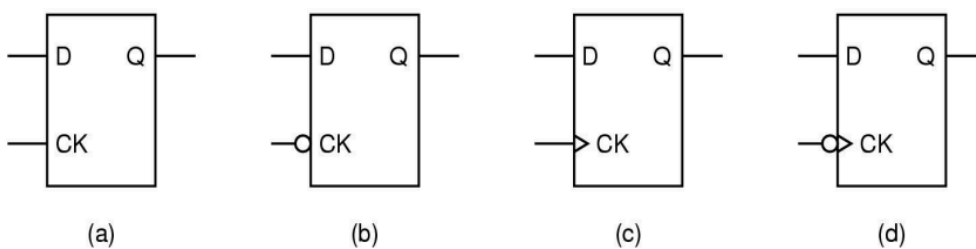
Eseguire il NOR del clock implica comunque del tempo perso. Mandando un segnale ed un not subito dopo, arriva solo per un istante il 1 del clock e viene quindi chiuso istantaneamente dal NOR lasciando un micro update in uscita.

### Flip-Flop di tipo D -

### Commutazione sul Fronte in Discesa

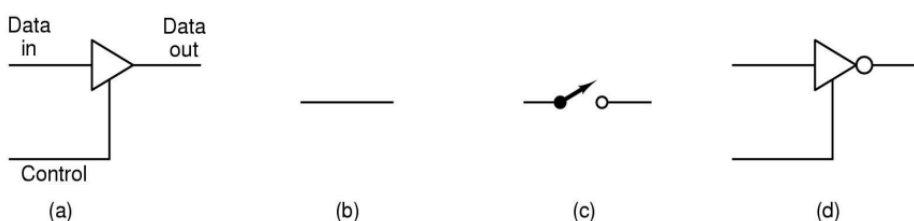


## LATCH E FLIP-FLOP - Rappresentazioni



- a) latch di tipo D      attivato con livello 1 del clock
- b) latch di tipo D      attivato con livello 0 del clock
- c) flip-flop di tipo D    attivato sul fronte di salita del clock
- d) flip-flop di tipo D    attivato sul fronte in discesa del clock

## BUFFER



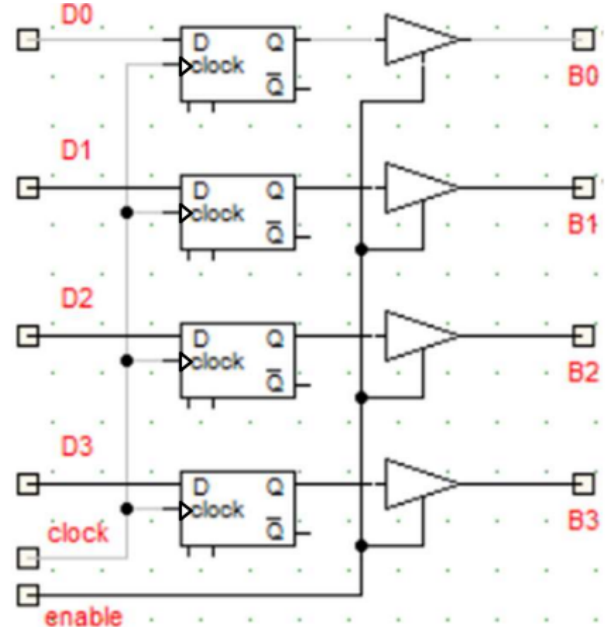
Abbiamo tre possibili stati:  
 b) Control = 1 → stream  
 c) Control = 0 → disconnect  
 d) Inverter

## Registri

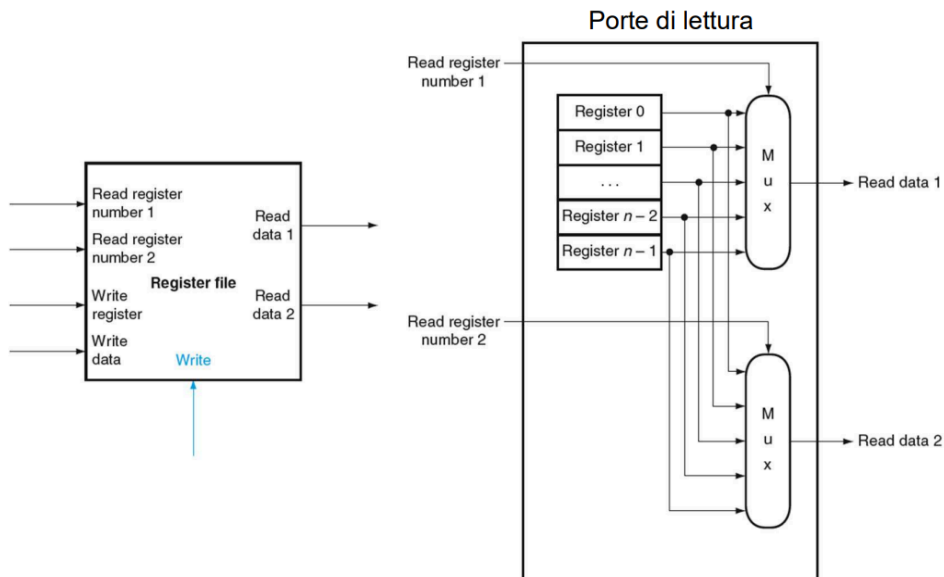
L'insieme di flip-flop D raggruppati insieme forma un registro

Il clock è collegato in ingresso a tutti i flip-flop del registro

L'ingresso enable permette di disconnettere (o riconnettere) il registro dal bus di output tramite buffer non invertente.



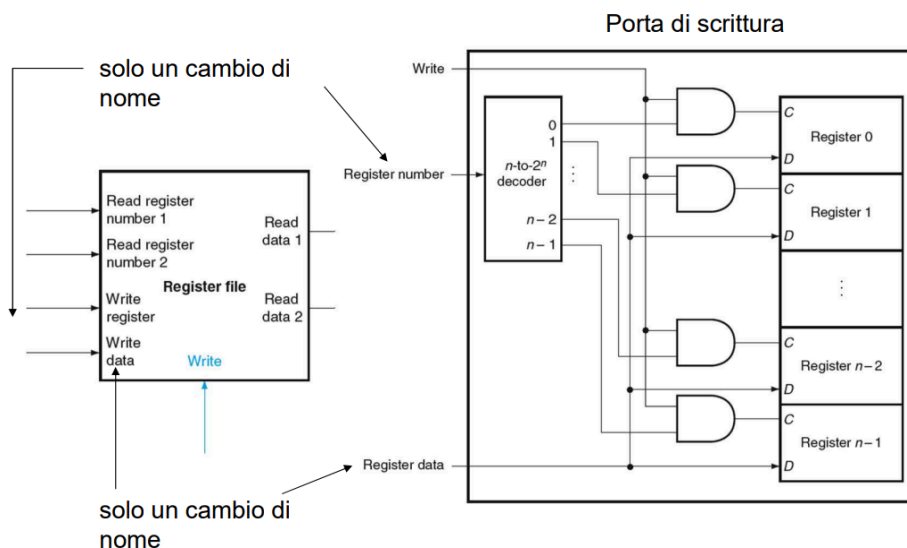
## Blocco di registry (register file)



Blocco di Registri  
(**register file**)

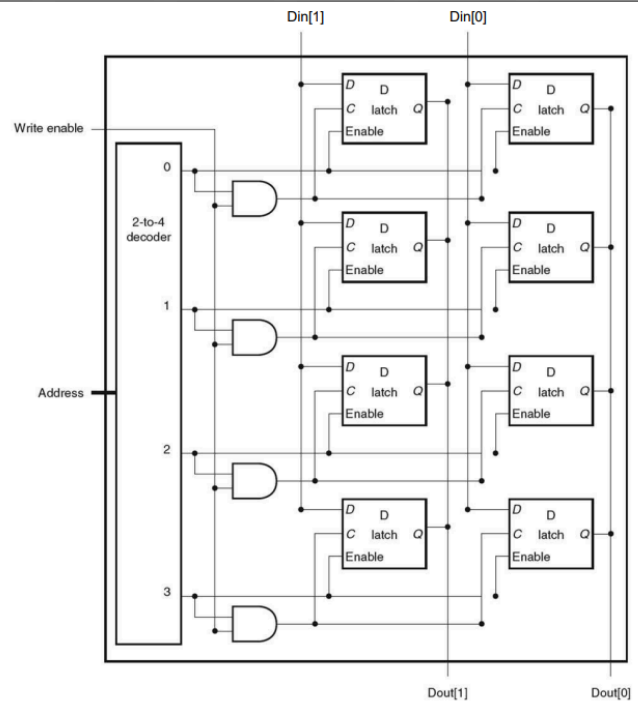
Fa uscire due output in  
quanto due sono gli  
operandi da usare  
nelle operazioni

## Blocco di registry (register file)



# Organizzazione della memoria

- memoria 4 x 2
- 5 linee di ingresso:
  - 2 per i dati di input
  - 2 per l'indirizzo
  - 1 per i bit di controllo:
    - Write enable
- 2 linee di uscita



## Chip di Memoria

- n linee di indirizzo corrispondono a  $2^n$  righe di flip-flop
- b linee di output corrispondono a b colonne di flip-flop
- I segnali possono essere attivi quando il livello è basso o alto (dipende dal chip)
- Matrici  $2^n \times b$ : seleziona della colonna e della riga

## Tipi di memoria: RAM - Random Access Memory

- SRAM: RAM statiche (flip-flop D), estremamente veloci, usate per realizzare Cache
- DRAM: RAM dinamiche (transistor con condensatore), vanno rinfrescati, offrono grandi capacità ma più lente
  - DRAM FPM: Fast Page Mode: organizzate in matrici
  - DRAM EDO: Extended Data Output: con semplice pipeline per l'output
  - SDRAM: Synchronous DRAM: dati e indirizzi controllati dallo stesso clock
  - SDRAM-DDR: Double Data Rate: leggono sia nel fronte in salita che discesa

## Tipi di memoria: ROM - Read Only Memory

Vengono usate per dati che non devono essere modificati

- PROM: Programmable ROM
- EPROM: Erasable PROM: memoria cancellabile mediante esposizione a UV
- EEPROM: Electrically Erasable PROM: cancellabile mediante impulsi elettrici (+ lente)
- FLASH: EEPROM cancellabile a blocchi

## **Analisi di Reti Sequenziali Sincrone**

TODO: fare sta parte che mi sto uccidendo a leggerla

**Se qualcun che legge avesse voglia di mandarmi gli appunti su sta parte, sarebbe una figata incredibile (si mi servono ancora, l'ho scritto il 30/05/2025)**

**Mi trovi su telegram: @JackChevalley**

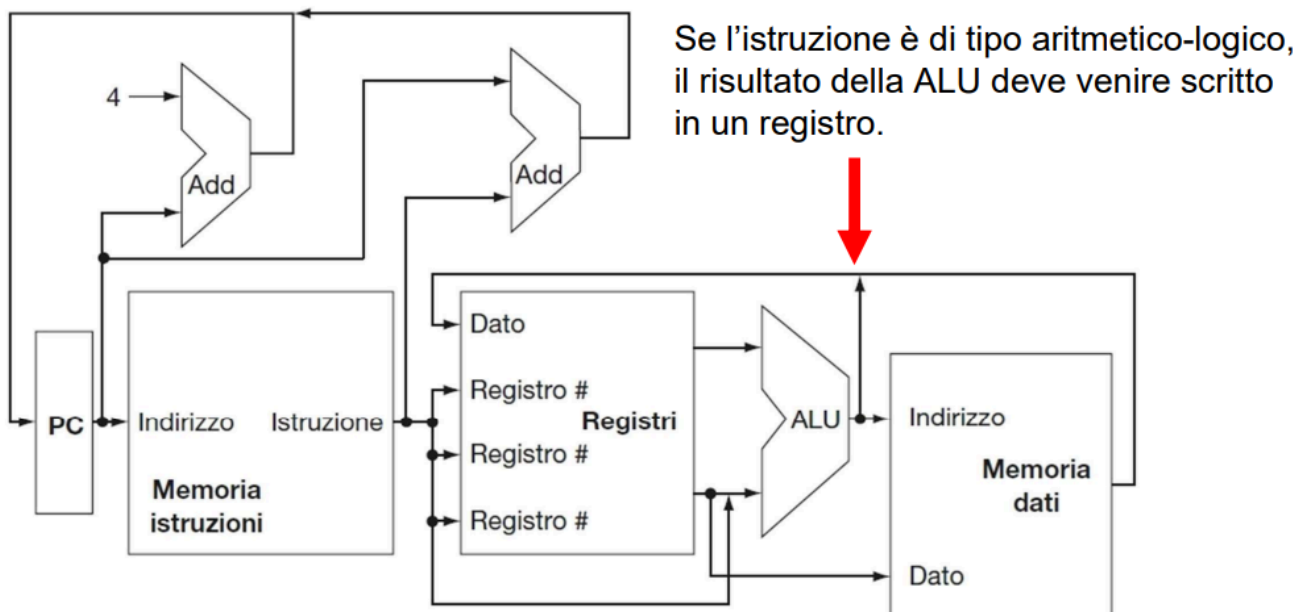
## DataPath nel processore RISC-V

Data Path: insieme di registri, l'unità aritmetico-logica e alcuni bus che li collegano.

Ciclo del data path: processo di far passare due operandi attraverso la ALU e memorizzare il risultato

## Fetch - Decode - Execute

1. Fetch
  - a. PC fornisce indirizzo della memoria delle istruzioni
2. Decode
  - a. ALU “capisce” il tipo di istruzione.
  - b. Avviene il decode per dell’istruzione e vengono caricate le componenti
3. Execute (una volta caricati gli operandi)
  - a. Eseguire un calcolo effettivo (operazioni aritmetico-logiche)
    - i. Il risultato viene scritto in un registro
  - b. Elaborare il contenuto per determinare un indirizzo di memoria (operazioni load/store)
    - i. Il risultato della ALU viene usato come indirizzo
  - c. Eseguire un confronto (salto condizionato)
    - i. Viene usata la ALU per calcolare l’indirizzo successivo



— questa immagine non è completa, è solo esemplificativa —

## Metodologia di Temporizzazione

Per determinare quando all'interno di un sistema sincrono un dato è valido e stabile, ovvero quando un cambio di stato è considerevole corretto, viene usato il clock sensibile ai fronti (*edge triggered*).

Il segnale di click determina quando gli elementi di stato scrivono la loro memoria interna.

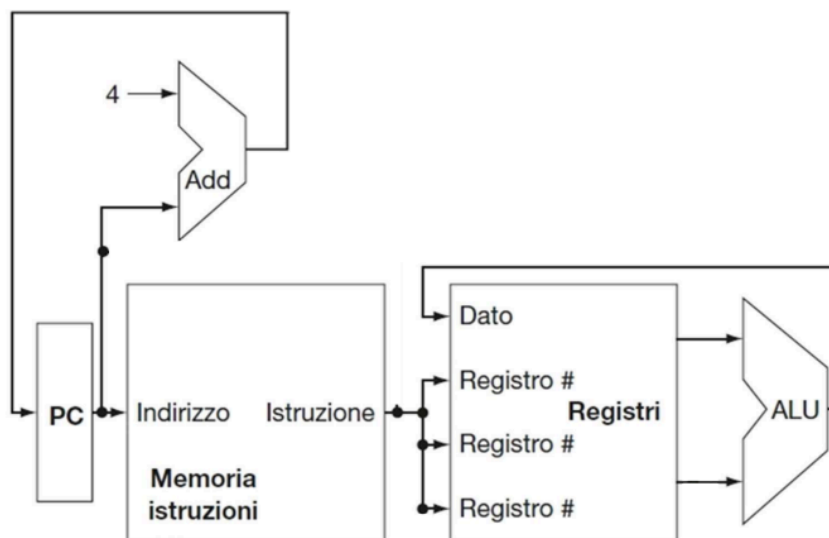
Questo implica che un dato debba diventare stabile prima che il clock ricominci il suo ciclo

# Realizzazione di un'Unità di Elaborazione

## Program Counter

Necessitiamo di alcune componenti per realizzare le sue operazioni:

1. Memoria Istruzioni
  - a. Qui vengono salvate le istruzioni del programma mappate mediante indirizzi.
  - b. Il register file eroga costantemente il contenuto dei registri forniti in input senza bisogno di alcun segnale di controllo.
  - c. La scrittura invece va esplicitamente indicata mediante il segnale di controllo
2. Program Counter (registro)
  - a. Registro utilizzato per memorizzare l'indirizzo dell'istruzione corrente
  - b. Il registro è (in questo caso) a 32 Bit
3. Adder (ADD component)
  - a. Un sommatore per incrementare PC e ottenere l'indirizzo successivo.
  - b. Tutte le istruzioni RISC-V sono rappresentate a 32 bit, quindi deve fare un jump di 4 Byte dopo ogni istruzione.
  - c. Nel caso di branch, l'adder calcola l'indirizzo corretto e non sommando solo 4 Byte



### Data Path per le istruzioni di TIPO R

La componente di memoria tiene "solo le istruzioni" in questo contesto. Immaginiamo due memorie separate per istruzioni e dati.



### Struttura della memoria di istruzioni

- 5 bit (indirizzi)
- Output bus 32 bit
- ALU Operations control: 4 bit

## Istruzioni Load (Tipo I) e Store (Tipo S)

Dobbiamo aggiungere due componenti agli schemi precedenti

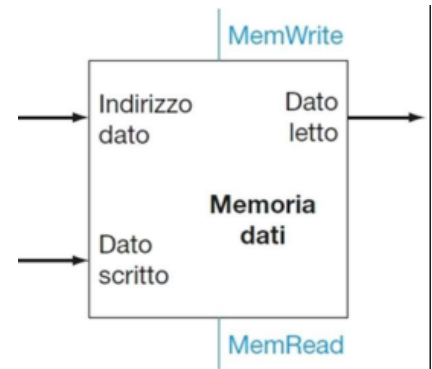
1. Memoria Dati (elemento di Stato)  
Necessita di un segnale di lettura / scrittura esplicito.  
Non è default read.  
La scrittura della memoria (in teoria) avviene sul fronte del segnale di clock.

Ingressi:

- a. Ingresso del dato  
→ `lw x1, offset(x2)`
- b. Il dato da scrivere  
→ `sw x1, offset(x2)`

Uscita:

- a. Il dato letto



2. Unità di Generazione delle Costanti (unità di estensione del segno)

Ha come obiettivi:

- a. Selezione dei 12 bit che compongono il campo immediato tra i 32 bit
- b. Estensione del segno (viene fornito un dato di 12 bit e va esteso a 32 tenendo il segno)
  - i. Viene propagato il bit più significativo  
 $-1 \rightarrow (12\text{bit}) 1111 1111 1111 \rightarrow (32\text{ bit}) 1111 1111 1111 \dots 1111$

## Istruzioni di Salto beq (Tipo SB)

Richiede il confronto di due valori che determina la successiva operazione.

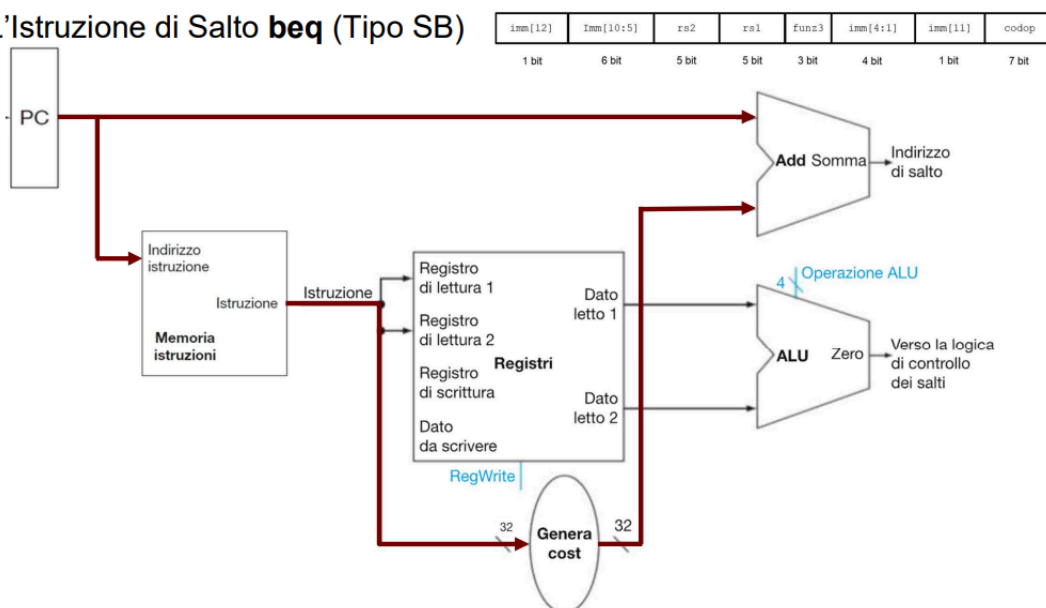
L'indirizzo di salto è relativo al PC e avviene come

$\text{new PC} = \text{indirizzo corrente} + \text{offset specificato}$

Con limite da -4096 a 4094 di offset e sempre in multipli di 2 in quanto l'offset viene sempre shiftato a sinistra di un bit di modo da avere l'allineamento contando quindi in effetti il numero di halfword e non lo spiazzamento in byte effettivo. Questo permette di avere salti + ampi, non ci serve descrivere un dato a metà quindi andiamo allineati (vedere la parte sull'allineamento).

Prima della somma l'indirizzo viene esteso a 32 bit con segno.

### L'Istruzione di Salto beq (Tipo SB)



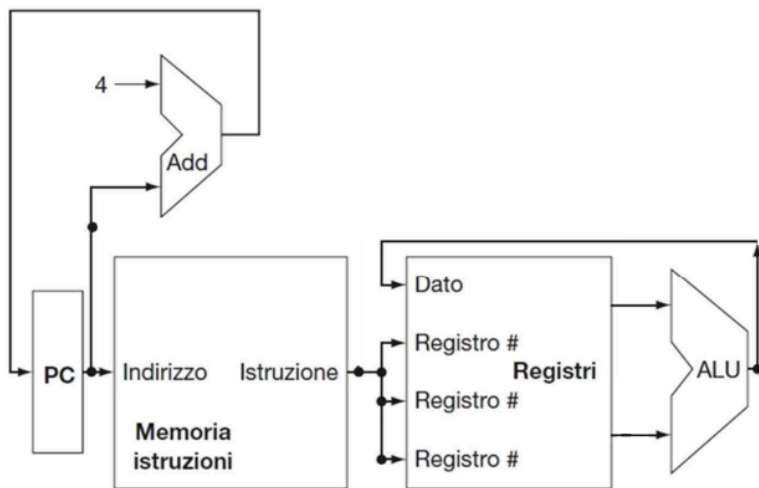
Branch not taken  
salto non preso  
continua al prox  
indirizzo

Branch taken  
salto preso  
calcola il prox  
indirizzo

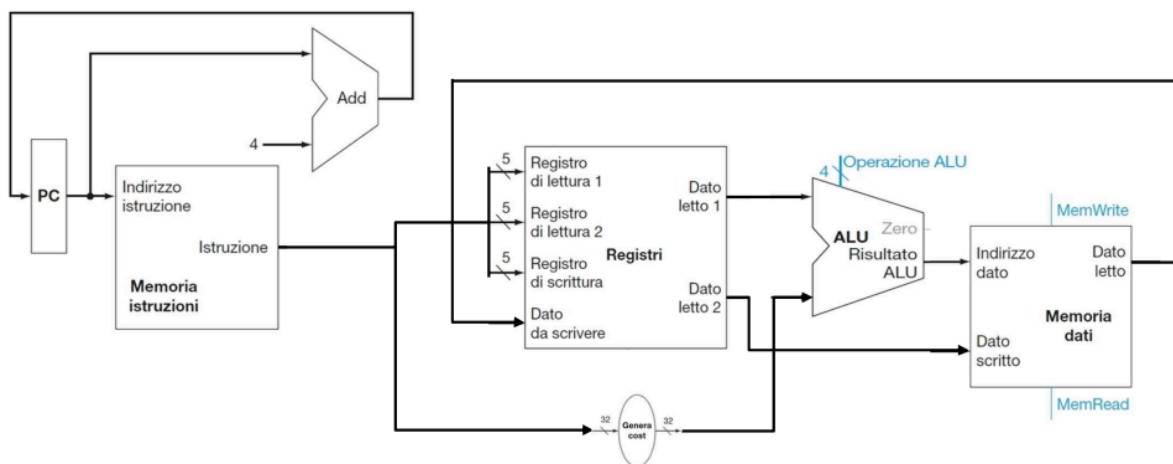
Salto  
Incondizionato  
Salta  
indipendentemente  
da tutto

Per confrontare fa eseguire il calcolo alla ALU e verifica l'uscita ZERO dell'ALU che indica se il risultato dell'operazione è zero. esce 1 se zero, esce 0 altrimenti. Torna indietro e verifica il next.

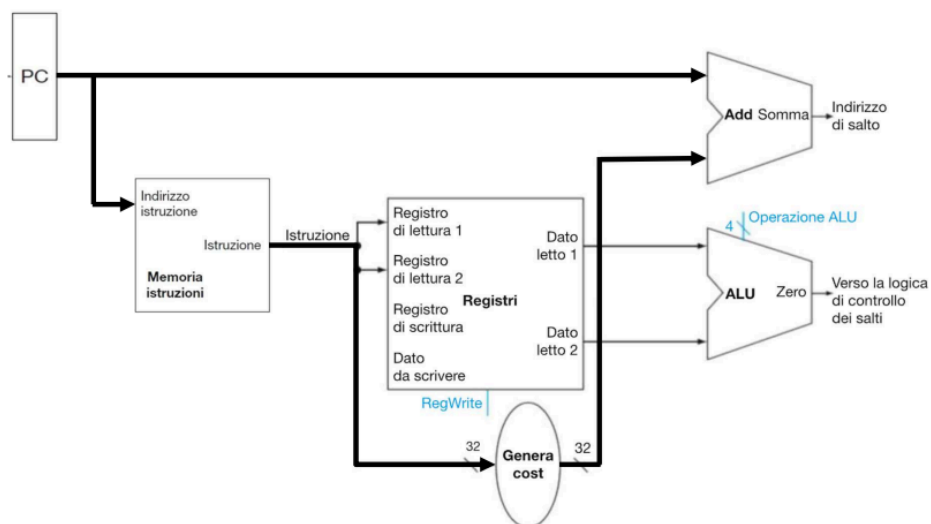
## Componente per tipo R



## Componente per tipo I, S



## Componente per il tipo SB

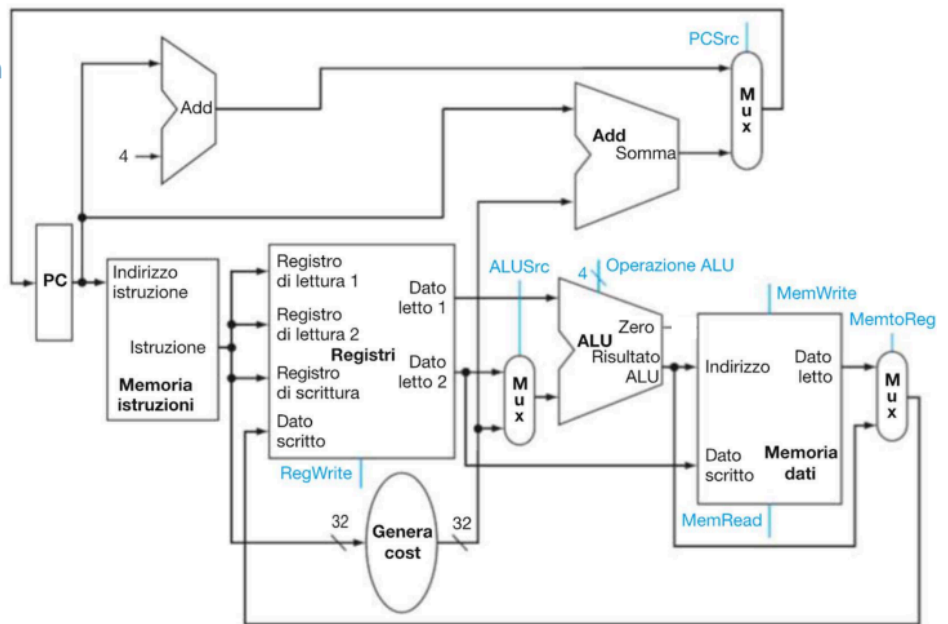


## Unificazione delle componenti - Unità di Elaborazione Unificata

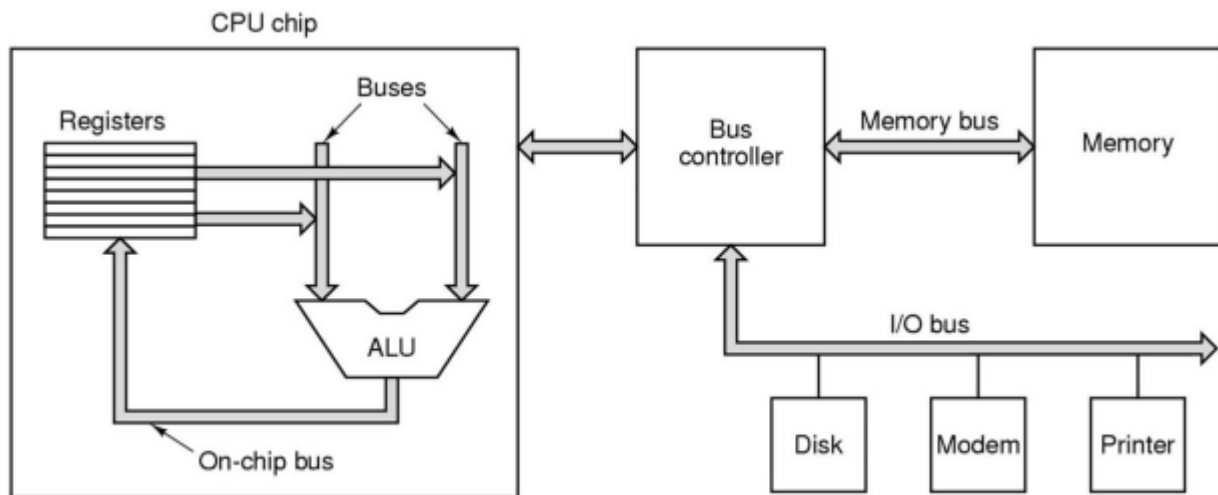
Per la scelta del percorso da seguire in base al tipo di dato, si usa un MultiPlexer.

Per integrare i salti condizionati, si aggiunge un altro MultiPlexer.

- Questa unità di elaborazione **esegue tutte le istruzioni in un singolo ciclo di clock.**
- Nessuna risorsa dell'unità potrà essere utilizzata più di una volta per ogni istruzione
- **Il controllo è fatto da un circuito combinatorio**
- Abbiamo duplicato diversi componenti (e.g., una memoria delle istruzioni separata da quella dei dati)



## Il BUS - Linee di Comunicazione



Il Bus è l'insieme di linee elettriche che collegano i moduli di un elaboratore.

Esistono vari tipi di bus:

- Bus di Sistema interconnette la CPU, le schede I/O e la memoria
- Bus interni al Chip mettono in comunicazione i moduli della CPU

Protocollo Bus: l'insieme delle regole

Le linee dei Bus possono essere:

- **linee di dati**: il numero di linee (larghezza bus)
- **linee di indirizzo**: permettono di individuare la sorgente / destinazione dei dati trasmessi
- **linee di controllo**: controllano l'accesso e l'utilizzo di dati e di indirizzi

Durante la progettazione dobbiamo tenere conto di:

- **Larghezza** del bus (numero di linee)  
Il numero di linee indica il numero massimo di dati trasferibili in una volta sola
- **Arbitraggio**: come scegliere quale dispositivo ha l'arbitraggio (uso) del bus attualmente
- **Funzionamento**: come avviene l'effettivo trasferimento?

### Connessioni di una CPU

"Piedini" sono gli attacchi elettrici della componente.

N è il numero di bit trasmessi contemporaneamente. Valori classici: 8, 16, 32, 64

- Indirizzo: n piedini corrispondono a  $2^n$  locazioni di memoria
- Dati: n piedini permettono di leggere / scrivere una parola di n bit con una sola operazione
- Controllo: regolano il flusso e la scansione dei dati verso e dal chip
  - controllo del bus
  - interrupt
  - arbitraggio dei bus
  - segnali del coprocessore
  - stato
  - varie

**Bus Driver**: chip specifico atto alla connessione di Bus e Chip effettivi per trasferimento dati

**Bus Receiver**: chip specifico atto a ricevere le richieste

**Bus Transceiver**: chip in grado di fare entrambe le cose, generico ad esempio per la CPU

Un dispositivo collegato ad un bus si dice

- **Attivo**: possono decidere di iniziare un trasferimento
- **Passivo**: rimangono in attesa di richieste

## Bus Clocking

—In sistemi moderni si fa tendenzialmente un mix—

**Bus Sincroni:** usano una linea di clock comune. Ogni operazione avviene ad un ciclo di clock, mai nel mezzo. Se l'operazione termina durante un ciclo di clock, aspetta.

Il bus sincrono deve adattarsi al dispositivo più lento.

La linea di clock è pilotata da un oscillatore.

- Linea **MREQ**: viene usata per comunicare inizio e fine di una comunicazione.  
Linea a 0-attivazione, quando il valore scende a 0 significa "in corso", poi torna ad 1

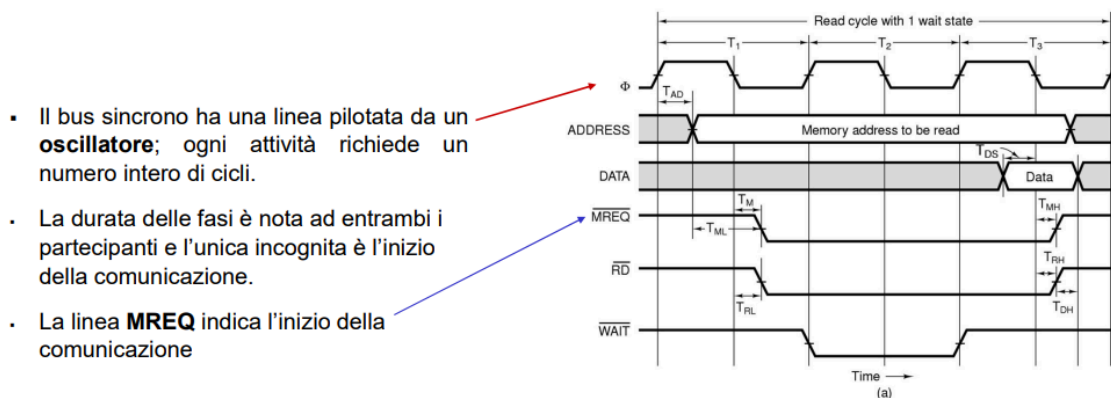
Vantaggi:

- Realizzazione device semplice
- Se la durata di un'operazione è fissa, non occorre una linea di wait

Svantaggi:

- Durata di un'operazione di comunicazione deve necessariamente avere una durata pari ad numero intero di cicli anche se necessita di meno.

La sequenza completa di lettura-scrittura è quella riportata qui sotto.

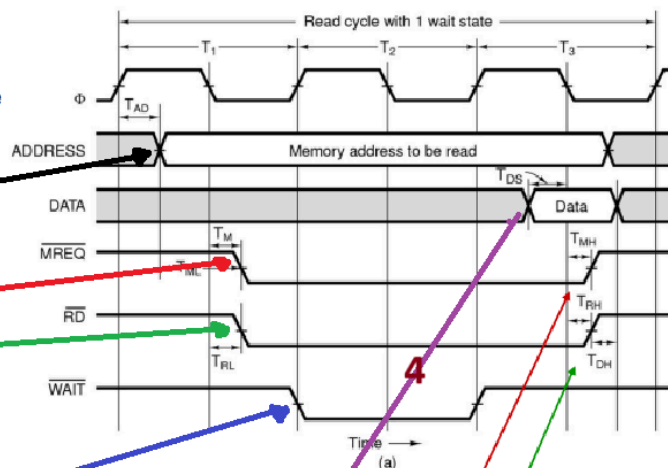


Supponiamo di avere un clock di 100 Mhz (ciclo di 10 nsec.) e che la lettura dalla memoria richieda 15 nsec da quando l'indirizzo è stabile

La **CPU** mette l'indirizzo sul bus, dopo un certo tempo necessario per stabilizzarsi ( $T_{AD}$ ), l'indirizzo rimane disponibile

A linee di indirizzo stabili la **CPU** attiva i segnali di richiesta di **inizio comunicazione** e il **segnale di lettura**

Per avvertire la CPU di non aspettare i dati, all'inizio del ciclo  $T_2$ , la **memoria** attiva il segnale di **attesa dati**



La **memoria** mette i dati sulla linea dati

Dopo aver letto i dati la **CPU** disattiva i segnali di **richiesta** e di **lettura**

**Bus Asincroni:** Non usano clock globale, ogni transazione può richiedere tempi diversi, sincronizzazione mediante segnali di handshaking.

Componenti:

- ADDRESS: rappresenta l'indirizzo di memoria da leggere, stabile nella transazione
- MREQ: la CPU lo attiva (va a 0) per indicare che vuole accedere alla memoria
- RD: la CPU lo attiva (va a 0) per indicare che vuole leggere e non scrivere
- MSYN: Segnale di sincronizzazione inviato dal master (CPU)  
La CPU lo attiva per indicare che è pronta, impostato tutto per i dati  
Questo segnale avvia effettivamente la richiesta
- DATA: La memoria una volta pronta inserisce i dati su queste linee
- SSYN: Segnale di sincronizzazione del destinatario (slave, cioè le memoria)

Sequenza completa (**FULL HANDSHAKE**)

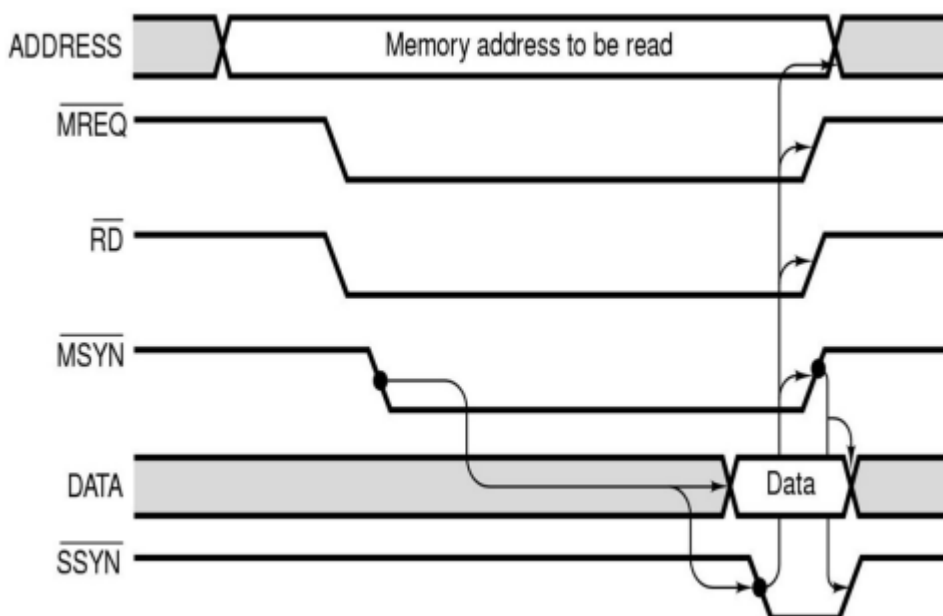
1. La CPU stabilisce l'indirizzo su ADDRESS e attiva MREQ e RD
2. Attiva MSYN → dice "ho fatto ora aspetto"
3. La memoria vede MSYN attivo → prepara i dati
4. Una volta che i dati sono pronti, attiva SSYN → dice "i dati sono sul bus"
5. La CPU legge i dati
6. La CPU disattiva MREQ, RD, MSYN → dice "ho finito di leggere i dati"
7. La memoria vede disattivarsi MSYN, disattiva SSYN → fine della transazione

Vantaggi

- Flessibilità: la durata di un'operazione è determinata dalla velocità della coppia di device

Svantaggi:

- Per completare l'operazione, sono sempre necessarie 4 azioni
- Occorre inserire nei device i circuiti necessari a rispondere opportunamente al protocollo

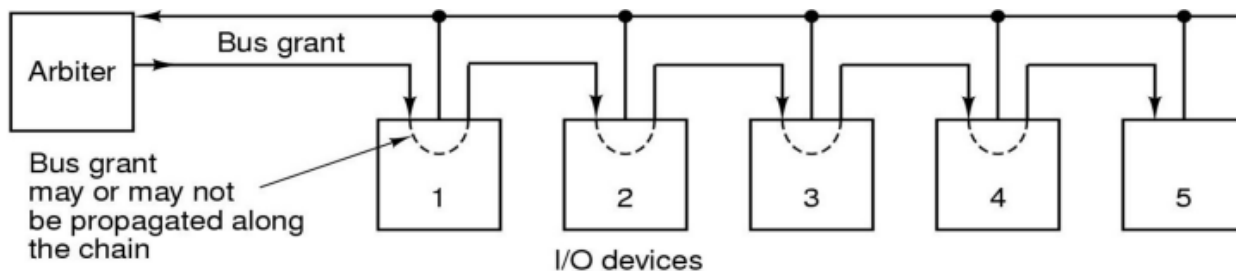


## Arbitraggio del Bus

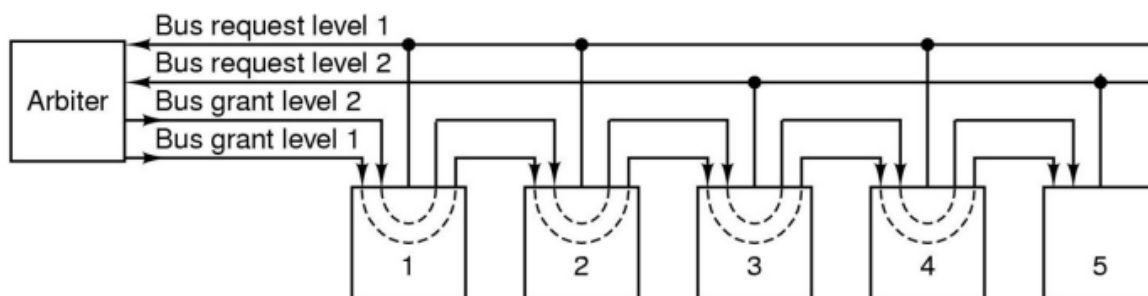
Gestione del bus in caso più dispositivi ne richiedessero contemporaneamente l'uso.

Arbitraggio Centralizzato: quando l'arbitro vede una richiesta, abilita la linea grant.

- **Daisy chaining:** Il primo dispositivo che la "ferma" prende la priorità. Avvantaggia quindi dispositivi vicini.



- **Linea di Acknowledge:** Come la Daisy chain ma aggiunge una linea di priorità. Vince non il più vicino ma il dispositivo indicato come prioritario.



Arbitraggio De-centralizzato:

Non c'è un arbitro centrale, ogni device decide in autonomia se ha la precedenza.

Può svilupparsi in diversi modi:

- Tante linee quanti i dispositivi, ogni dispositivo osserva le linee prima di effettuare una richiesta
- Soluzione a tre linee:
  - bus request: indica che qualcuno vuole usare il bus
  - busy: segnala che il bus è attualmente occupato
  - linea di arbitraggio: servono a decidere chi vince

Fasi spiegate:

1. Ogni dispositivo mette il proprio ID sulle linee di arbitraggio
2. Confronta il proprio ID con quello degli altri
3. Se vede un ID più alto (priorità maggiore), si ritira
4. Il dispositivo con l'ID più alto vince e prende il controllo del bus

## Bus Paralleli e Seriali

Bus paralleli: usano molteplici linee di comunicazione in parallelo per trasmettere contemporaneamente tutti i bit di una parola. La trasmissione presenta limiti come:

- Interferenza tra linee (crosstalk)
- Sincronizzazione dei dati
- Costi elevati
- Distanze Limitate

Bus seriali: serializzano/codificano il dato prima di trasmetterlo. Trasmettono effettivamente un solo bit alla volta ma in modo estremamente veloce. Sono ad esempio le USB 3.x, Thunderbolt, ...

# Memoria: Organizzazione e Gerarchia

## La Memoria Principale

- Store / Storage
- Memoria come insieme di celle
  - La cella è la minima unità indirizzabile
- Tutte le celle hanno la stessa dimensione
- Ogni cella è identificata da un indirizzo
- Se un calcolatore ha n celle allora gli indirizzi vanno da 0 a n-1
- Il numero di bit nell'indirizzo determina il numero massimo delle celle indirizzabili
- La memoria consiste in  $2^n$  indirizzi, possiamo suddividerla in **blocchi di k Byte**

$$\text{Numero di blocchi realizzabili} \rightarrow M = 2^n / k$$

I bit vengono ordinati secondo diverse strategie

### - big endian:

la numerazione va da sinistra a verso destra

Usato in:  
SPARC,  
mainframe IBM

|   |      |   |
|---|------|---|
|   | 7    | 0 |
| 0 | \$1D |   |
| 1 | \$2C |   |
| 2 | \$3B |   |
| 3 | \$4A |   |
| 4 | 'C   |   |
| 5 | 'I   |   |
| 6 | 'A   |   |
| 7 | 'O   |   |
|   | byte |   |

|   |      |      |   |   |   |
|---|------|------|---|---|---|
|   | 15   | 8    | 7 | 0 | 0 |
| 0 | \$1D | \$2C |   |   | 1 |
| 2 | \$3B | \$4A |   |   | 3 |
| 4 | 'C   | 'I   |   |   | 5 |
| 6 | 'A   | 'O   |   |   | 7 |
|   | Word |      |   |   |   |

|   |           |      |      |      |  |   |
|---|-----------|------|------|------|--|---|
|   | 31        |      | 16   | 15   |  | 0 |
| 0 | \$1D      | \$2C | \$3B | \$4A |  | 3 |
| 4 | 'C        | 'I   | 'A   | 'O   |  | 7 |
|   | Long-word |      |      |      |  |   |

Organizzazione big endian: numero \$1D2C3B4A E STRINGA "CIAO"

### - little endian

la numerazione va da destra verso sinistra

Usato in:  
Intel,  
RISC-V

|   |      |   |
|---|------|---|
|   | 7    | 0 |
| 0 | \$1D |   |
| 1 | \$2C |   |
| 2 | \$3B |   |
| 3 | \$4A |   |
| 4 | 'C   |   |
| 5 | 'I   |   |
| 6 | 'A   |   |
| 7 | 'O   |   |
|   | byte |   |

|   |      |      |   |   |   |
|---|------|------|---|---|---|
|   | 15   | 8    | 7 | 0 | 0 |
| 1 | \$2C | \$1D |   |   | 0 |
| 3 | \$4A | \$3B |   |   | 2 |
| 5 | 'I   | 'C   |   |   | 4 |
| 7 | 'O   | 'A   |   |   | 6 |
|   | Word |      |   |   |   |

|   |           |      |      |      |  |   |
|---|-----------|------|------|------|--|---|
|   | 31        |      | 16   | 15   |  | 0 |
| 3 | \$4A      | \$3B | \$2C | \$1D |  | 0 |
| 7 | 'O        | 'A   | 'I   | 'C   |  | 4 |
|   | Long-word |      |      |      |  |   |

Organizzazione little endian: numero \$4A3B2C1D E STRINGA "CIAO"

## CPU e Memoria, La CACHE

La CPU è molto più veloce di una memoria, ci vogliono diversi cicli CPU per ottenere il dato richiesto alla memoria.

Viene introdotta una piccola quantità di memoria molto veloce (e costosa) interna alla CPU: **la Cache**. All'interno della cache troveremo le parole più usate di modo che siano più vicine e rapide. Troviamo sia Dati che Istruzioni nella cache.

L'accesso alla memoria non avviene a caso ma secondo principi di località:

- **Località Temporale:** principio secondo cui se si accede a una determinata locazione di memoria, è molto probabile che vi si acceda di nuovo dopo poco tempo.
- **Località Spaziale:** principio secondo cui se si accede a una determinata locazione di memoria, è molto probabile che si acceda anche alle locazioni contigue a breve.

La cache consiste in C linee di k Byte (blocchi)

C è molto minore del numero di blocchi in Memoria (M)

Quando un dato viene richiesto, se presente in cache, viene passato dalla cache. → hit rate  
Altrimenti viene letto e contemporaneamente trasmesso a cache e CPU → miss rate

### Come tenere traccia dei dati in Cache ? Mappatura Diretta

La mappatura diretta consiste nel "Ogni dato della memoria presente in cache, è presente esattamente (al più) solo una volta"

**Per determinare in quale linea della cache inserire il blocco** si usa la seguente formula:

$$\text{Linea} = \text{Indirizzo del blocco} \% \text{Numero di blocchi della cache}$$

Ha come aspetti positivi la semplicità di sviluppo e la velocità di calcolo della linea.

Ha come problematiche il fatto che due dati potrebbero sovrascrivere costantemente sulla stessa riga, viene definito conflict miss.

**Per determinare se il dato è presente in cache** invece, ogni linea della cache ha:

- Un BIT DI VALIDITA' per dire se contiene dati validi. Quando accendiamo il PC tutti i bit di validità sono = 0, quando inseriamo o aggiorniamo un dato, passano ad 1
- un TAG che identifica quale blocco di memoria è stato salvato in quella linea

Quando si accede ad un dato:

1. Si calcola la linea
2. Si confronta il tag della linea con quello dell'indirizzo richiesto
3. Se coincidono ed è valido → hit
4. Altrimenti → miss

$$\text{Numero di Linee} = \text{Dimensione totale dei dati} / \text{Dimensione di ciascun blocco}$$

### Come tenere traccia dei dati in Cache ? Completamente Associativa

Un blocco della memoria principale può essere scritto in una qualsiasi locazione della cache.

La ricerca del dato va effettuata su tutti gli elementi della cache (mediante un comparatore, aumenta i costi), viene confrontato ogni tag fino a trovare il dato.

Funziona solo per un numero limitato di blocchi.

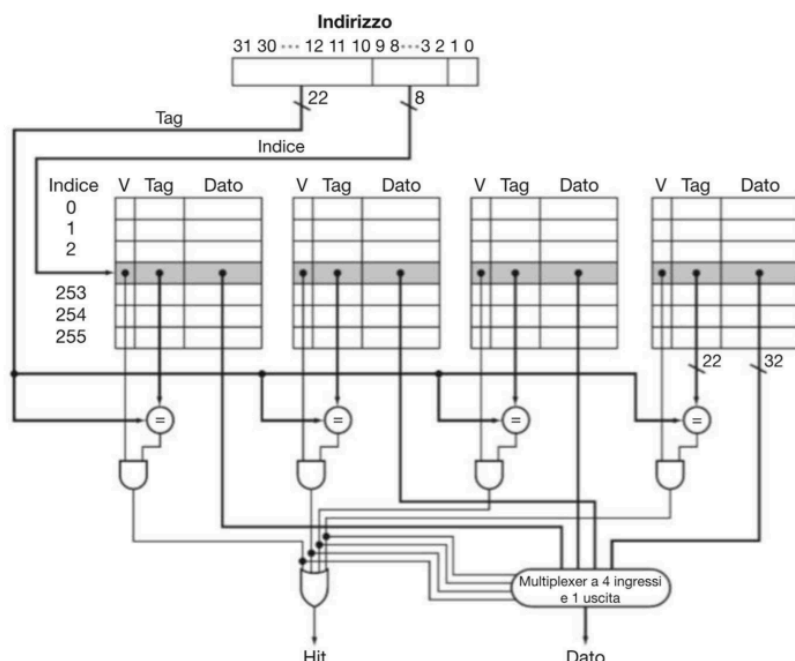
## Come tenere traccia dei dati in Cache (veramente) ?

## Set-Associativa

Un bel mix delle precedenti. La memoria è divisa in “set”, ogni blocco di memoria può andare in un solo set. La ricerca avviene quindi non sulla cache intera ma solo nelle linee del set definito.

$\text{Set} = \text{Numero Blocco} \% \text{Numero di set della cache}$

Viene sostituito il blocco usato meno di recente (LRU)



### Cache set-associativa a quattro vie:

- quattro comparatori
- 1 multiplexer con 4 ingressi e 1 uscita.

I comparatori individuano il blocco di quale insieme, se esiste, corrisponde al tag dell'indirizzo.

L'uscita dei comparatori viene utilizzata per selezionare il dato da uno dei quattro blocchi della linea selezionata, attraverso un multiplexer pilotato dai segnali di hit dei diversi blocchi.

## Gestione delle MISS - Miss della Cache (sia dati che istruzioni)

1. Inviare il valore originale del Program counter alla memoria
2. Comandare alla memoria principale di eseguire un'operazione di lettura e attendere che la memoria completi la lettura
3. Scrivere la parola che proviene dalla memoria nella posizione opportuna del blocco della cache, aggiornare il campo “tag” scrivendovi i bit più significativi dell'indirizzo presi direttamente dalla ALU e impostare a 1 il bit di validità
4. far ripartire l'esecuzione dell'istruzione dall'inizio, ripetendo la fase di fetch che questa volta troverà l'istruzione all'interno della cache

## Gestione della scrittura

### - Write-through

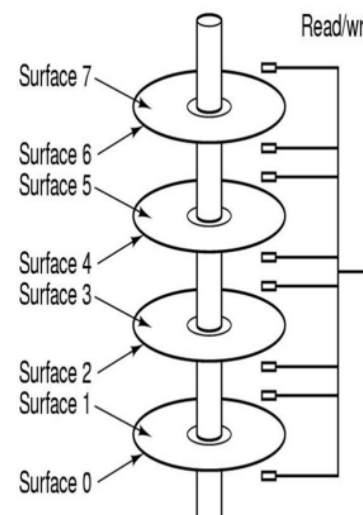
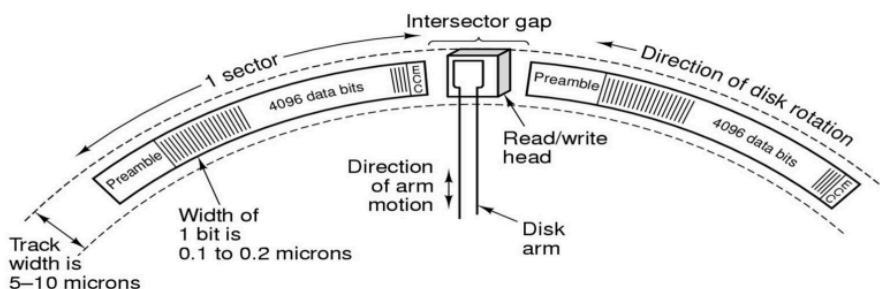
- Ad ogni scrittura in cache, si modifica il valore anche in memoria
- Se il dato da scrivere non è in cache, si mette in cache e poi in memoria
- non ha ottime prestazioni

### - Write-back

- Il dato viene scritto solo in locale
- La scrittura al livello inferiore avviene solo quando il blocco nella cache va rimpiazzato
- è complesso da implementare

## La Memoria Secondaria: Memoria di Massa - Dischi Magnetici

Si tratta di memorie di massa, più lente di una RAM ma decisamente più grandi ed economiche (ordine dei ms). Sono composte da alluminio(o vetro) con rivestimento magnetizzabile.



### Componenti fisiche:

- Piatto: disco in alluminio rivestito da uno strato magnetico (tra 3, 9cm)
- Testina di lettura / scrittura: si muove radialmente per accedere ai dati nelle varie tracce
- Altezza di volo: la testina "vola" a 0.5μm dal disco

### Componenti Logiche:

- Traccia: cerchio concentrico su cui sono scritti i dati
- Settore: posizione di una traccia (generalmente almeno 512 Byte)
- Dimensione: larghezza su traccia di 1 Bit: tra 0.1 e 0.2 micron
- Cilindro: insieme di tracce allineate verticalmente su più piatti
- Gap: spazio tra settori, usato per la sincronizzazione

### Componenti Settore:

- Un preambolo per la sincronizzazione
- I dati
- Un codice di correzione degli errori (es. Hamming, Reed-Solomon)
- Un Gap per separarlo dal settore successivo

### Scrittura / Lettura

- Posizionamento in corrispondenza dell'indirizzo desiderato
- Tempo di accesso:
  - spostamento testina (seek time) +
  - tempo perché il disco ruoti nel posto giusto (latency time) +
  - tempo di lettura dell'informazione

### Tipologie di dischi:

- Winchester
- Floppy disk
- IDE, EIDE: controllore integrato nel drive
- ATA-3: aumento della velocità di trasferimento e dello spazio degli indirizzi
- ARAPI-7: non aumenta la velocità di trasferimento ma la dimensione del connettore

### Interfacce dei dischi:

- SCSI (Small Computer System Interface): garantisce alta velocità di trasferimento
- RAID (Redundant Array of Inexpensive / Independent Disk): insieme di dischi, organizzati a livelli e da un controllore. Si comporta come un disco normale ma il controllore sparge i dati / li duplica per avere ridondanza

## Input - Output

La comunicazione con i device di I/O avviene attraverso i controllori che:

- trasformano i comandi della CPU in segnali elettrici per le periferiche
- trasformano i segnali delle periferiche in dati per la CPU

Ogni controllore ha al suo interno dei registri identificati da un indirizzo che può:

- appartenere allo stesso insieme di indirizzi di memoria (memory mapped I/O)
- essere un indirizzo dedicato (isolated I/O)

I registri dei controllori contengono

- dati (ingresso ed uscita)
- comandi (dalla CPU alla periferica)
- informazioni sullo stato della periferica (dalla periferica alla CPU)

## Le Interrupt

Gli Hardware Interrupt sono cambiamenti nel flusso di controllo legati alla gestione dei periferici e non al programma stesso, vanno sempre dai dispositivi alla CPU.

Quando viene mandato un interrupt, il device attiva una linea del BUS direttamente connessa a CPU. L'interrupt interrompe il programma in atto e trasferisce il controllo ad un gestore di interrupt che esegue le azioni appropriate.

Terminata la gestione dell'interrupt, lo stato dell'esecuzione deve tornare esattamente come era prima dell'interrupt stesso tornando al vecchio programma eseguito. Questa si chiama "trasparenza". Sono azioni asincrone rispetto al programma.

### Azioni HardWare all'interruzione:

1. Il controller attiva una linea di interrupt del bus di sistema
2. Quando la CPU è pronta a gestire l'interruzione, manda un ack sul bus
3. Il controller invia un intero sulle linee dati (valore di interrupt)
4. La CPU preleva il vettore di interrupt dal bus e lo salva
5. La CPU impila PC e altri registri di stato sullo stack
6. Il vettore di interrupt è usato come indice di memoria per trovare il valore di PC a cui si trova il gestore di quel tipo di interruzione
7. Si modifica PC, e a volte si evita che un'altra interruzione possa interrompere la gestione dell'interruzione in corso di esecuzione

### Azioni SoftWare all'interruzione:

1. Il codice ISA di gestione delle interruzioni salva tutti i registri (stato del programma che è stato interrotto)
2. Si possono leggere tutte le informazioni dal buffer del dispositivo
3. Se c'è stato un errore di I/O lo si gestisce
4. Le variabili ptr e count vengono aggiornate, se ci sono altri caratteri da visualizzare si copia il carattere puntato da ptr nel buffer del controller del dispositivo
5. Eventuali altri ack di fine trattamento interruzione
6. Ripristino dei registri
7. Esecuzione di una apposita istruzione di "Return from Interrupt - RETI" che ripristina modalità e stato della CPU

## Gestione di più Interrupt generate da dispositivi diversi

Ci sono due modalità da analizzare:

- Il gestore delle interruzioni disabilita le interruzioni mentre ne sta gestendo una
- Si definiscono delle priorità tra i dispositivi che possono generare delle interrupt. Se ne arriva una di priorità maggiore, interrompe la gestione attuale. La priorità dipende dalla criticità del dispositivo che manda l'interrupt

## Trap, Eccezione, Interrupt Sincroni

Sono chiamate a procedure automatiche che possono verificarsi per

- Condizioni eccezionali
- Istruzioni apposite per richiedere servizi di base del sistema operativo

Essendo dipendenti dal programma (e non assolute), sono sincrone al programma.

Gestione della trap: procedura di gestione della richiesta di trap la cui posizione (indirizzo) è memorizzata in una locazione fissa a cui il programma salta in caso di trap.

Le eccezioni sono individuate dall'hardware in quanto, ad esempio, l'overflow viene verificato in parallelo ad altre procedure.

## Gestione dell'overflow con e senza Trap

- Senza Trap: l'hw setta un bit in un apposito registro per segnalare overflow, il programmatore come istruzione successiva va a controllare il valore del bit e, se segnato come overflow, valuta il salto condizionato da fare.
- Con Trap: l'hw rileva l'overflow e genera una trap che modifica il flusso di controllo ad una locazione prefissata

Registri Speciali RISC-V

- **SEPC**: l'indirizzo del codice in cui si verifica l'eccezione
- **SCAUSE**: la causa dell'eccezione

L'eccezione permette alle applicazioni di avere accesso al calcolatore in modo controllato, attraverso che sta nel sistema operativo.

- **Modalità Supervisor (kernel mode)**: durante l'eccezione il calcolatore esegue codice del sistema operativo
- **Modalità User (user mode)**: le applicazioni vengono eseguite in modalità user, senza poter fare accesso alle risorse privilegiate hardware del calcolatore

- Interrupt
  - Eccezione causata da eventi esterni
  - Asincrona rispetto all'esecuzione del programma
- Errore
  - Eccezione causata da eventi interni
  - Sincrona rispetto al programma
- Environment Call
  - Eccezione sincrona causata da richiesta di un Servizio di Sistema
- Environment Break
  - Eccezione sincrona causata da motivi diagnostici o debugging

Il gestore dell'eccezione viene eseguito in Kernel Mode, ecco i metodi di implementazione:

STVEC: registro speciale composto da BASE + MODE

- **Salto diretto all'indirizzo della routine di gestione**

Viene usato il registro speciale STVEC che contiene l'indirizzo del gestore.

Il gestore quando parte, legge SCAUSE e valuta come gestire l'eccezione

PC = BASE

MODE = 00

- **Vettore di Interruzione**

PC = MEM [base + cause \* 4]

Ovvero calcola in base ad una tabella di conversione dove SCAUSE assegna a cause un valore intero per calcolare l'indirizzo in memoria.

Lo stato viene salvato su: stack (push), registri ausiliari (shadow registers), registri speciali.

## Modalità di I/O:

### - I/O programmato con busy waiting

La CPU controlla attivamente lo stato del I/O e lo stato della periferica ispezionando in un ciclo il bit del registro di stato del controllore fino a quando non segnala di essere pronto per un nuovo comando di I/O.

#### Lettura:

1. il controllore della tastiera alla pressione di un tasto, porta ad 1 il bit di stato
2. il controllore trascrive il codice ASCII associato al tasto nel registro buffer
3. la CPU controlla il bit di stato finchè non lo trova con valore 1
4. Quando vale 1, prende il dato dal buffer e lo sposta nel suo registro. Rimette a 0 lo stato

#### Scrittura:

1. la CPU attende che il dispositivo sia pronto ad accogliere un carattere (bit Ready = 1)
2. la CPU invia il carattere sul display buffer
3. il controllore azzerà il bit Ready e visualizza il contenuto del buffer (bit Ready = 0)
4. il controllore segnala di essere pronto a ricevere un nuovo comando (bit Ready = 1)

### - I/O controllato da Interrupt

Il dispositivo quando ha finito genera un segnale che avverte la CPU di aver completato il proprio lavoro (interrupt).

La CPU abilita l'interrupt mettendo ad 1 un opportuno bit, altrimenti ignora

### - I/O in DMA (Direct Memory Access)

Si tratta di un I/O programmato ma svolto da una componente apposita con accesso diretto ai bus di memoria.

La CPU e il DMA si contendono l'uso del bus

|          | SD | LW | ADD/SUB | BEQ |
|----------|----|----|---------|-----|
| ALUsrc   | 1  | 1  | 0       | 0   |
| RegWrite | 0  | 1  | 1       | 0   |
| MemRead  | 0  | 1  | 0       | 0   |
| MemWrite | 1  | 0  | 0       | 0   |
| Branch   | 0  | 0  | 0       | 1   |

ALUsrc impostato a 1 quando il 2° operando è immediato, se è dato da un registro allora è 0

RegWrite è 1 quando si scrive su un registro, ad esempio ADD o LW lo fanno

MemRead è 1 quando si legge in memoria, LW lo fa quando carica dato

MemWrite è 1 quando si scrive in memoria, SD quando scrive dato

Branch è 1 quando vi è un confronto

Il circuito che di solito viene allegato non è fondamentale alla esecuzione dell'es, come anche i valori delle variabili. Basta vedere la tipologia di istruzione

| ALU_op | Operazione | Significato                        |
|--------|------------|------------------------------------|
| 0000   | AND        | $a \& b$                           |
| 0001   | OR         | $\sim a$                           |
| 0010   | ADD        | $a + b$                            |
| 0110   | SUB        | $a - b$                            |
| 0111   | SLT        | $a < b$ (signed, Set on Less Than) |
| 1100   | NOR        | $\sim(a$                           |